tcb/box tcb/boxSect
tcb/box/begin0 tcb/box/begindefault
tcb/box/begindefault
tcb/box/end0 tcb/box/enddefault
tcb/box/enddefault
tcb/box/title tcb/box/titleCaption
tcb/box/title0 tcb/box/titledefault para/semantictcb/box/title tcb/box/titledefault
tcb/box/draw2 tcb/box/drawdefault 2
tcb/box/drawdefault
tcb/box/init0 tcb/box/initdefault minipage/beforenoopminipage/afternoop tcb/box/initdefault
tcb/box/upper0 tcb/box/upperdefault minipage/beforetag/dfltminipage/aftertag/dflt tcb/box/upperdefault
tcb/box/lower0 tcb/box/lowerdefault minipage/beforetag/dfltminipage/aftertag/dflt tcb/box/lowerdefault
tcb/drawing/init0tcb/drawing/initdefaulttcb/drawing/initdefault

Formalizing the HIPAA Privacy Rule (and Beyond) in Soufflé Datalog

A Comprehensive Guide to the ComplianceGPT Datalog Engine
Covering HIPAA §§164.502–164.524 and Multi-Regulation Extension
(GDPR · GLBA · CCPA · COPPA · SOX)

Version 3 — Updated with Multi-Regulation Support, System Architecture, and Benchmarking

Prepared for Priscilla
ComplianceGPT Project — Stony Brook University

April 2026

Abstract

This document provides a detailed, end-to-end explanation of how the HIPAA Privacy Rule was formalized in Soufflé Datalog, and how that formalization was extended to five additional regulations. The HIPAA coverage includes §164.502 (General Rules), §164.506 (Treatment, Payment, and Health Care Operations), §164.508 (Authorization Required), §164.510 (Opportunity to Agree or Object), §164.512 (Uses and Disclosures Without Authorization), §164.514 (Other Requirements), and §164.524 (Individual Access). Starting from the source PDF formalization in first-order temporal logic, we describe every design decision, type declaration, hierarchy, helper predicate, and rule translation. The document covers the architecture of the compliance checker, the ADT-based explanation tree mechanism for provenance, the handling of stratified negation, oracle predicates, grounding issues, and testing methodology.

Version 3 adds: (1) a system architecture overview for the full ComplianceGPT pipeline (LLM1 → Soufflé → LLM2), including all four strategies (Baseline, RAG, Formal, Agentic) and their components; (2) formalization details for GDPR (Arts. 6 and 17), GLBA (§§313.14), CCPA (§§1798.100, 1798.120), COPPA (§§312.4–312.5), and SOX (§§302, 404); (3) the unified bridge interface used by all six regulation engines; (4) benchmarking methodology using the multi-regulation QA datasets; and (5) the RAG knowledge base design for each regulation. The intended audience is a student or researcher learning to understand and extend this formalization.

Contents

1 Introduction to Soufflé Datalog

1.1 What is Datalog?

Datalog is a declarative logic programming language that is a subset of Prolog. Unlike Prolog, Datalog:

- **Always terminates**: there are no function symbols (no infinite terms), so the set of derivable facts is always finite.
- **Uses bottom-up evaluation**: the engine starts from known facts and derives all possible conclusions, rather than searching top-down from a query.
- **Has set semantics**: duplicate tuples are automatically eliminated.
- **Supports stratified negation**: negation is allowed under a structural discipline that prevents paradoxes.

A Datalog program consists of:

1. **Facts** (the Extensional Database, or EDB): ground tuples that are asserted as true.
2. **Rules** (the Intensional Database, or IDB): implications of the form `head :- body.` that derive new tuples from existing ones.
3. **Queries / Output directives**: which relations to materialize and export.

1.2 What is Soufflé?

Soufflé (Systematic, Ontological, Underpinning Framework for Facts, Logic, and Evaluation) is a high-performance Datalog engine developed at Oracle Labs and the University of Sydney. Key features relevant to our project:

- **Typed relations**: every relation has a declared schema with typed columns (`.type` and `.decl`).
- **Algebraic Data Types (ADTs)**: tagged unions that allow building recursive data structures (like explanation trees) as first-class values.
- **C preprocessor**: `#define`, `#include`, and `#ifdef` directives are supported, enabling macros and modular file organization.
- **Stratified negation**: the `!` operator for negation-as-failure, enforced at compile time.
- **Provenance / Explain**: the `-t explain` flag provides an interactive proof-tree REPL for debugging derivations.
- **Aggregation**: `count`, `sum`, `min`, `max` with grouping.
- **Disjunction in rule bodies**: the `;` operator for logical OR within a rule body.

1.3 Installing Soufflé

On macOS with Homebrew:

```
brew install souffle
```

On Ubuntu/Debian:

```
sudo apt-get install souffle
```

Verify the installation:

```
souffle --version
```

1.4 Running Soufflé Programs

Soufflé compiles and evaluates `.dl` files directly. There is no separate build step.

```
# Run a program, writing output CSVs to a directory
souffle -D <output_dir> <program>.dl

# Run the HIPAA compliance checker
souffle -D output_hipaa hipaa_main.dl

# Interactive provenance / proof-tree REPL
souffle -t explain hipaa_main.dl

# ncurses TUI for large proof trees
souffle -t explore hipaa_main.dl

# View the stratification (precedence) graph
souffle --show=precedence-graph hipaa_main.dl

# Generate an HTML debug report (requires Graphviz)
souffle -r report.html hipaa_main.dl
```

1.5 Core Syntax Summary

Here is a minimal Soufflé program to illustrate the key syntax:

```
1  // Type declarations
2  .type Person <: symbol
3
4  // Relation declaration
5  .decl parent(child: Person, par: Person)
6  .decl ancestor(descendant: Person, anc: Person)
7
8  // Facts (EDB)
9  parent("alice", "bob").
10 parent("bob", "carol").
11
12 // Rules (IDB)
13 ancestor(X, Y) :- parent(X, Y).                  // base case
14 ancestor(X, Z) :- parent(X, Y), ancestor(Y, Z).  // recursive case
15
16 // Output directive
17 .output ancestor
```

Listing 1: Minimal Soufflé example

Key syntax elements:

- `.type Name <: symbol` declares a named subtype of the built-in `symbol` type.
- `.decl relation(col1: Type1, col2: Type2)` declares a relation with a typed schema.
- `head :- body.` is a rule. The head is derived when all body literals are satisfied.
- `,` in a rule body means conjunction (AND).
- `;` in a rule body means disjunction (OR).
- `!literal` is negation-as-failure (the literal must not hold).
- `.output relation` writes the relation's contents to a CSV file.
- `#include "file.dl"` includes another Datalog file (C preprocessor).
- `#define MACRO value` defines a preprocessor macro.

1.6 The Explain Feature

The `-t explain` flag starts an interactive REPL after evaluation. You can ask for proof trees:

```
souffle -t explain hipaa_main.dl
> explain is_disclosure_allowed("mercy_hospital", "dr_smith",
    "patient_alice", "msg_001", "blood-test-results", "treatment", _)
```

This displays a tree showing exactly which rules and facts combined to derive the tuple. For non-existent tuples, `explainnegation` provides guided exploration of why no derivation exists.

1.7 Stratification Visualization

Soufflé partitions rules into *strata* (layers) based on the dependency graph. To visualize this:

```
souffle --show=precedence-graph hipaa_main.dl
```

This outputs the dependency graph showing which relations depend on which others, with negative edges marked. The engine evaluates strata bottom-up: stratum 0 first (base facts), then stratum 1 (positive rules), then stratum 2 (rules that negate stratum-1 relations), and so on.

2 Project Overview

2.1 What is HIPAA?

The Health Insurance Portability and Accountability Act (HIPAA) is a United States federal law enacted in 1996. Title II of HIPAA established national standards for the privacy and security of individually identifiable health information. The **HIPAA Privacy Rule** (45 CFR Part 160 and Subparts A and E of Part 164) regulates the use and disclosure of Protected Health Information (PHI) by covered entities and their business associates.

2.2 What is the Privacy Rule?

The Privacy Rule governs:

- Who may access PHI (covered entities, business associates, individuals).

- Under what conditions PHI may be used or disclosed (treatment, payment, healthcare operations, authorization, public interest, etc.).
- What safeguards must be in place (minimum necessary standard, business associate agreements, notice of privacy practices).
- Individual rights (access to records, accounting of disclosures, restriction requests).

2.3 Formalized HIPAA Sections: §§164.502, 506, 508, 510, 512, 514, and 524

The compliance checker formalizes seven core sections of the HIPAA Privacy Rule. The first two (§164.502 and §164.506) form the general framework; the remaining five cover specific categories of use and disclosure.

§164.502 — General Rules (fully formalized): The gatekeeper section and the top-level entry point of the compliance checker. It specifies that a covered entity may not use or disclose PHI *except* as permitted or required by the Privacy Rule. All permitted disclosure pathways are enumerated: subsections (a)(1)(i)–(vi) for permitted uses, (a)(2) for required disclosures (to the individual and to the Secretary), and a series of constraints and special cases: minimum necessary in (b), restriction agreements in (c), de-identified information in (d), business associates in (e), personal representatives in (g), incidental uses in (a)(1)(i), and whistleblower/crime victim provisions in (j). In Soufflé, §164.502 is the final rules file included before the top-level `is_disclosure_allowed` predicate.

§164.506 — Treatment, Payment, or Health Care Operations (TPO) (fully formalized): The implementation specification for TPO disclosures, referenced by §164.502(a)(1)(ii). When control passes to this section, it has three main parts: (a) standard permitted uses requiring no authorization, (b) consent for TPO (optional for most covered entities), and (c) the implementation specification with five sub-rules covering: treatment disclosures by any covered entity, payment operations, health care operations between entities with a treatment relationship, organized health care arrangement disclosures, and the case where a covered entity that is not a health plan or provider may engage in health care operations only when both entities have or had a relationship with the subject.

§164.508 — Uses and Disclosures for Which an Authorization is Required (formalized): Governs disclosures that require the individual's written authorization. The section distinguishes uses that *always* require authorization — psychotherapy notes (a)(2) and disclosures for marketing (a)(3) or sale of PHI (a)(4) — from the general case where authorization is required absent another permission pathway. The formalized rules check `obtained_authorization_164_508` and attribute/purpose predicates to classify psychotherapy-note and marketing-purpose disclosures correctly.

§164.510 — Uses and Disclosures Requiring an Opportunity to Agree or Object (formalized): Covers two specific scenarios that require the covered entity to give the individual an opportunity to agree or object before the disclosure: (a) facility directories (name, location, general condition, religious affiliation) and (b) disclosures to family members, close friends, and others involved in the individual's care or payment for care. The formalized rules check the `family-or-friend-care` and `facility-directory` purpose classes, and verify that the individual has not objected (oracle predicate `individual_agreed_510`).

§164.512 — Uses and Disclosures for Which Individual Authorization or Opportunity to Agree or

The "public interest" section. It enumerates twelve specific categories of permitted disclosures that override the general authorization requirement, including: required by law (b), public health activities (b)(1), abuse/neglect reporting (c), health oversight (d), judicial/administrative proceedings (e), law enforcement (f), decedents (g), organ donation (h), research (i), serious threat to health or safety (j), specialized government functions (k), and workers' compensation (l). The formalized rules map these to purpose classes (e.g., `public-health`, `law-enforcement`, `research`) and verify the associated conditions (e.g., IRB waiver for research, imminent danger for (j)).

§164.514 — Other Requirements Relating to Uses and Disclosures of PHI (formalized): Covers de-identification of PHI and the limited data set provisions. Under (a)–(b), information is not PHI once it satisfies the Expert Determination or Safe Harbor de-identification standard. Under (e), a covered entity may use or disclose a "limited data set" (dates, city, zip code, ages retained; 18 direct identifiers stripped) for research, public health, or health care operations without individual authorization, provided a data use agreement is in place. The formalized rules check the `is_deidentified` and `is_limited_dataset` oracle predicates.

§164.524 — Access of Individuals to Protected Health Information (formalized): Grants individuals the right to inspect and obtain a copy of their own PHI held in a designated record set. The right is subject to narrow enumerated exceptions: psychotherapy notes, compiled for legal proceedings, correctional/law enforcement scenarios, research where access is temporarily suspended, Privacy Act-exempt systems, and situations where a licensed health care professional determines that access could endanger the life or safety of the individual or another person. The formalized rules check the `is_individual_requesting_own` predicate and the six denial grounds, producing ALLOWED when the requester is the subject and no denial ground applies.

Stub sections (conservative denial). Sections §§164.520 (notice of privacy practices), 164.522 (rights to restrict disclosures), 164.528 (accounting of disclosures), 164.530 (administrative requirements), 164.504 (organizational requirements), and 160.C (enforcement) are declared in `hipaa_stubs.dl` with empty bodies. Under the closed-world assumption this means any disclosure that would require these sections for permission is classified as DENIED until full formalization is added.

2.4 The Goal: A Compliance Checker

The project aims to build an automated compliance checker that, given a proposed disclosure (sender, receiver, subject, message, attribute type, purpose), determines:

1. Whether the disclosure is **allowed** under the HIPAA Privacy Rule.
2. **Why** it is allowed — a structured explanation tree citing the specific sections and sub-sections that permit it.
3. If it is **denied**, the closed-world assumption provides the reason: no permission pathway matched.

The source formalization lives in `HIPAA_FORMALIZATION.pdf` (pages 30–122), written in first-order temporal logic. Our task is to translate this into Soufflé Datalog with ADT-based explanation trees.

3 Architecture and File Organization

3.1 File Overview

The compliance checker is organized into 10 files, included in dependency order by the entry point `hipaa_main.dl`:

File	Purpose
`hipaa_types.dl`	Type declarations (`Principal`, `Role`, `Attribute`, `Purpose`, `Message`, `TimePoint`, `Rel`), preprocessor macros (`ARGS`, `DECL_ARGS`), and the `Expl` ADT for explanation trees.
`hipaa_hierarchies.dl`	Role hierarchy (`role_isa`, `role_subtype`), attribute hierarchy (`attr_isa`, `attr_subtype`), purpose hierarchy (`purpose_isa`, `purpose_subtype`), transitive closures, `has_role`, `belongs_to`, and organizational/relationship predicates.
`hipaa_macros.dl`	Helper predicates translating the PDF's macros: `is_covered_entity`, `is_phi`, `is_for_tpo`, `is_personal_representative`, oracle predicates (`has_authority_to_act`, `believes_minimum_necessary`, etc.).
`hipaa_stubs.dl`	Empty `.decl` declarations for sections not yet formalized (§164.508, §164.510, §164.512, §164.514, §164.520, §164.522, §164.524, §164.528, §164.530, §164.504, §160.C). Conservative default: no tuples, no permissions.
`hipaa_164_506.dl`	§164.506 rules: TPO disclosure pathways (a), (b), (c)(1)–(c)(5).
`hipaa_164_502.dl`	§164.502 rules: general rules (a)(1)(i)–(vi), (a)(2)(i)–(ii), (b) minimum necessary, (c) restriction agreements, (d) de-identified info, (e) business associates, (h) provider/plan communication, (i) notice consistency, (j) whistleblower/crime victim.
`hipaa_top.dl`	Top-level `is_disclosure_allowed`, denial tracking (`is_disclosure_denied`), and `.output` directives.
`hipaa_facts.dl`	Test fact database: principals, roles, organizational relationships, messages, oracle predicates, and 10 disclosure attempt scenarios.
`hipaa_main.dl`	Entry point: `#includes` all files in dependency order.
`run_hipaa.sh`	Shell script: cleans output, runs Soufflé, and pretty-prints results.

Table 1: File organization of the HIPAA compliance checker.

3.2 Dependency Order

The `#include` order in `hipaa_main.dl` is carefully chosen:

```
// 1. Type declarations and ADT explanation tree
#include "hipaa_types.dl"
// 2. Role, attribute, and purpose hierarchies with transitive closure
#include "hipaa_hierarchies.dl"
// 3. Helper predicates (macros from the PDF formalization)
#include "hipaa_macros.dl"
// 4. Stub declarations for sections not yet formalized
#include "hipaa_stubs.dl"
// 5. 164.506: Treatment, payment, healthcare operations
```

```
#include "hipaa_164_506.dl"
// 6. 164.502: General rules (references 164.506)
#include "hipaa_164_502.dl"
// 7. Top-level rule and output directives
#include "hipaa_top.dl"
// 8. Fact database (test scenarios)
#include "hipaa_facts.dl"
```

Listing 2: hipaa_main.dl — include order

The ordering ensures that:

- Types are declared before they are used in relation declarations.
- Hierarchies and helper predicates are defined before rules reference them.
- Stubs for unformalized sections are declared before §164.506 and §164.502 reference them (in negation or delegation).
- §164.506 is defined before §164.502 because §164.502(a)(1)(ii) delegates to §164.506.
- The top-level rule aggregates all permission pathways from §164.502.
- Facts come last so all relation declarations are in scope.

Design Decision

Although Soufflé does not strictly require declaration order (it does a two-pass analysis), we maintain a logical ordering for human readability and to match the structure of the HIPAA regulation itself.

4 General Principles of Modeling

Before diving into the type system and individual rule translations, it is helpful to understand the recurring architectural patterns that appear throughout the HIPAA formalization. These principles apply uniformly to every section of the Privacy Rule and should guide the formalization of sections beyond §164.502 and §164.506.

4.1 High-Level Rule Organization Pattern

Every HIPAA section is modeled using a two-level pattern: a **top-level disjunction** that ORs together all sub-sections, and a **hierarchical decomposition** where each sub-section further decomposes into its own sub-rules.

Top-level disjunction. Each HIPAA section (e.g., §164.502) is modeled as a top-level OR of its subsections. In Soufflé, this means one rule per subsection, all with the same head predicate:

```
.decl permitted_by_164_502(DECL_ARGS, e: Expl)

permitted_by_164_502(ARGS, E) :- permitted_by_164_502_a(ARGS, E).
permitted_by_164_502(ARGS, E) :- permitted_by_164_502_c(ARGS, E).
permitted_by_164_502(ARGS, E) :- permitted_by_164_502_d(ARGS, E).
permitted_by_164_502(ARGS, E) :- permitted_by_164_502_e(ARGS, E).
permitted_by_164_502(ARGS, E) :- permitted_by_164_502_h(ARGS, E).
```

```
permitted_by_164_502(ARGS, E) :- permitted_by_164_502_i(ARGS, E).
permitted_by_164_502(ARGS, E) :- permitted_by_164_502_j(ARGS, E).
```

Listing 3: Top-level disjunction for §164.502 (from hipaa_164_502.dl)

Multiple rules with the same head predicate produce a union (logical OR) in Datalog. If *any* subsection permits the disclosure, the top-level predicate holds.

Hierarchical decomposition. Each subsection further decomposes. For example, §502(a) splits into §502(a)(1) (permitted) and §502(a)(2) (required). Then §502(a)(1) splits into (i) through (vi):

```
// 502(a) -> (a)(1) permitted OR (a)(2) required
permitted_by_164_502_a(ARGS, ...) :- permitted_by_164_502_a_1(ARGS, E).
permitted_by_164_502_a(ARGS, ...) :- required_by_164_502_a_2(ARGS, E).

// 502(a)(1) -> (i) through (vi)
permitted_by_164_502_a_1(ARGS, E) :- permitted_by_164_502_a_1_i(ARGS, E).
permitted_by_164_502_a_1(ARGS, E) :- permitted_by_164_502_a_1_ii(ARGS, E).
permitted_by_164_502_a_1(ARGS, E) :- permitted_by_164_502_a_1_iii(ARGS, E).
permitted_by_164_502_a_1(ARGS, E) :- permitted_by_164_502_a_1_iv(ARGS, E).
permitted_by_164_502_a_1(ARGS, E) :- permitted_by_164_502_a_1_v(ARGS, E).
permitted_by_164_502_a_1(ARGS, E) :- permitted_by_164_502_a_1_vi(ARGS, E).
```

Listing 4: Hierarchical decomposition (from hipaa_164_502.dl)

Naming convention. Predicate names follow the pattern `permitted_by_164_XXX_Y_Z` where `XXX` is the section number, `Y` is the subsection letter, and `Z` is the paragraph number or Roman numeral. For required disclosures, the prefix is `required_by_` instead.

Rule hierarchy visualization. The following trees show the decomposition structure for the two formalized sections:

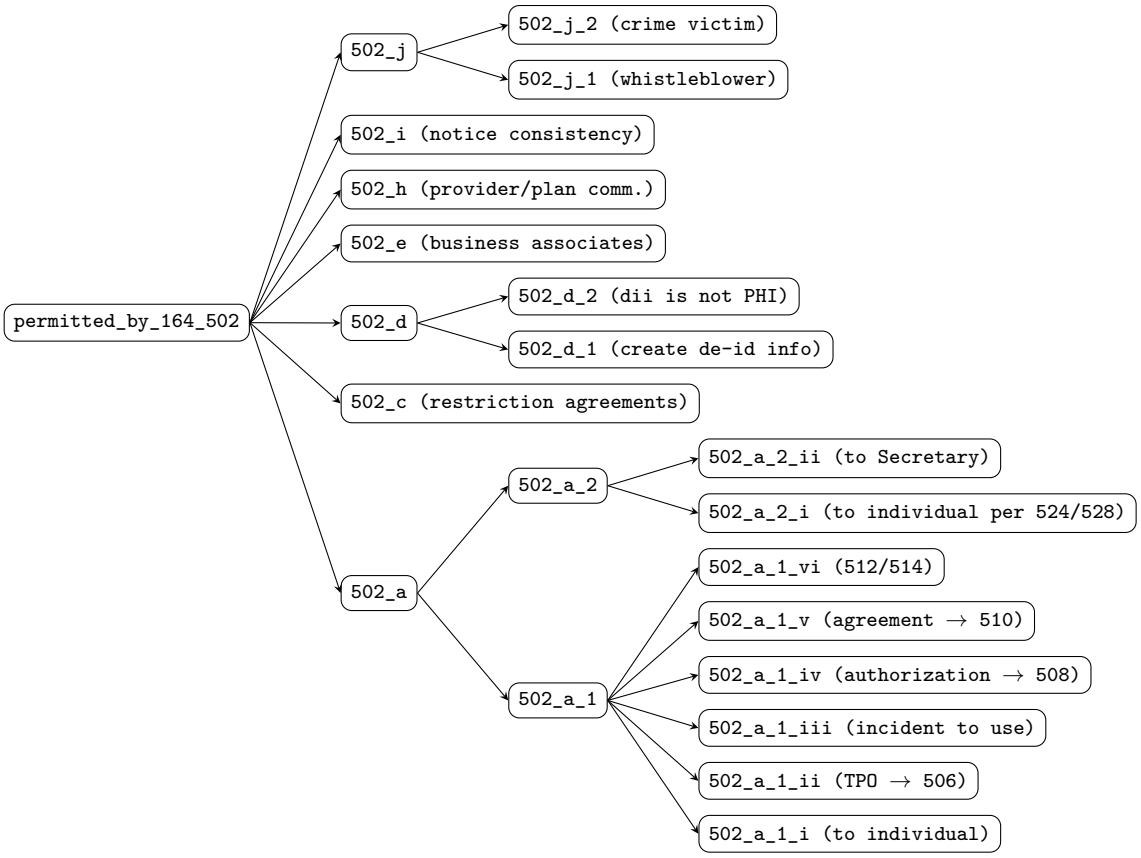

Figure 1: Rule hierarchy for §164.502.

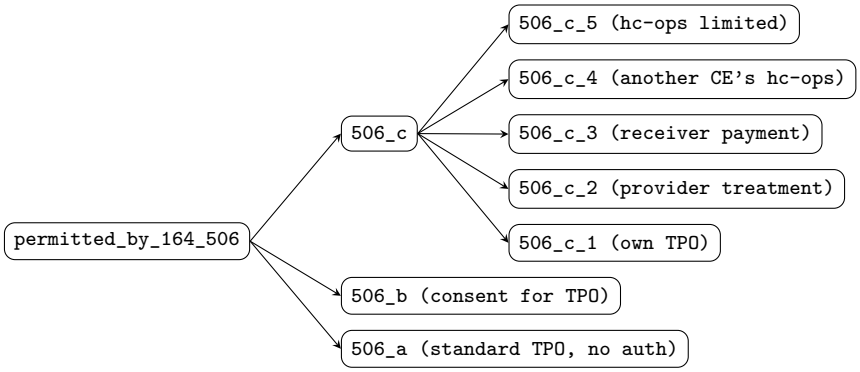

Figure 2: Rule hierarchy for §164.506.

4.2 Three Types of Norms

The source PDF formalization (DeYoung et al.) classifies HIPAA norms into three categories. Recognizing which category a norm belongs to determines how it is translated into Soufflé.

Positive norms (φ^+): Rules that **permit** a disclosure. Each positive norm becomes a Soufflé rule whose head is a `permitted_by_` or `required_by_` predicate, and whose explanation tree uses `$Leaf` (for base permissions) or `$Step1`/`$Step2` (for delegated permissions).

Example — §164.502(a)(1)(i) is a positive norm permitting disclosure to the individual:

```
permitted_by_164_502_a_1_i(ARGS,
    $Leaf("164.502(a)(1)(i): disclosure to the individual")) :-
    is_covered_entity(p1),
    is_phi(t),
    msg_contains(m, q, t),
    known_purpose(u),
    (p2 = q ; is_personal_representative(p2, q)).
```

Negative norms (φ^-): Rules that **block** or **constrain** a disclosure. These appear in the PDF as negated conditions (¬, written `\+` in Prolog). In Soufflé, they are translated using stratified negation (`!predicate`).

Example — §164.506(a) negates the authorization requirement from §164.508:

```
permitted_by_164_506_a(ARGS, ...) :-
    !require_authorization_by_164_508(ARGS),  // negative norm
    is_covered_entity(p1),
    is_for_tpo(u),
    permitted_by_164_506_c(ARGS, E).
```

Constraints: Rules that impose conditions but are *not* independent permission paths. They are enforced as guards within other rules rather than standing alone. The most important example is the minimum necessary standard (§502(b)):

```
// NOT a top-level permission path -- it's a guard:
minimum_necessary_satisfied(ARGS) :-
    believes_minimum_necessary(ARGS).
minimum_necessary_satisfied(ARGS) :-
    excluded_164_502_b_2(ARGS).
```

The constraint is used *inside* other rules (e.g., § 502(a)(1)(iii) requires `minimum_necessary_satisfied(ARGS)` in its body) but does not appear as a clause of `permitted_by_164_502`.

Design Decision

Distinguishing these three categories is critical when reading the PDF. A constraint that is mistakenly modeled as a positive norm would create a spurious permission path. A positive norm mistakenly modeled as a constraint would be invisible at the top level.

4.3 Exception Handling Patterns

HIPAA is full of exceptions — "except when," "this does not apply to," "notwithstanding." Three distinct patterns appear in the formalization:

Pattern 1: Exception-as-negation. A base rule includes `!excluded_predicate(...)` in its body. The excluded predicate collects all exception conditions via separate rules. If any exception condition holds, the negation fails and the base rule does not fire.

Example — §502(e)(1)(i) permits disclosure to a business associate, *except* in cases listed in (e)(1)(ii):

```
// Base rule with negated exception
permitted_by_164_502_e(ARGS, ...) :-
    is_covered_entity(p1),
    is_business_associate_of(p2, p1),
    is_phi(t), msg_contains(m, q, t),
    is_for_ba_purpose(u),
    satisfactory_assurances(p1, p2, q, t, u),
    !excluded_164_502_e_1_ii(ARGS).       // exception check

// The exclusion collects three cases:
// (A) Provider treatment disclosures
excluded_164_502_e_1_ii(ARGS) :-
    is_covered_entity(p1), is_health_care_provider(p2),
    is_for_treatment(u), is_phi(t), msg_contains(m, q, t).
// (B) Group health plan disclosures per 164.504(f)
excluded_164_502_e_1_ii(ARGS) :-
    (is_health_plan(p1) ; has_role(p1, "health-insurance-issuer")),
    permitted_by_164_504_f(ARGS), is_phi(t), msg_contains(m, q, t).
// (C) Government program providing public benefits
excluded_164_502_e_1_ii(ARGS) :-
    has_role(p1, "government-entity"), is_health_plan(p2),
    known_purpose(u), is_phi(t), msg_contains(m, q, t).
```

Listing 5: Exception-as-negation pattern (from hipaa_164_502.dl)

Pattern 2: Exception-as-separate-path. An exception creates an entirely independent permission path rather than negating a base rule. This is used when the "exception" is really an alternative way to satisfy a requirement.

Example — the minimum necessary exceptions (§502(b)(2)) do not negate the base minimum necessary rule. Instead, they provide an alternative satisfaction via `excluded_164_502_b_2`:

```
// Minimum necessary is satisfied if EITHER:
// (1) the entity believes it is minimum necessary, OR
// (2) an exception applies
minimum_necessary_satisfied(ARGS) :-
    believes_minimum_necessary(ARGS).
minimum_necessary_satisfied(ARGS) :-
    excluded_164_502_b_2(ARGS).

// Exception (b)(2)(i): treatment disclosures to providers
excluded_164_502_b_2(ARGS) :-
    is_covered_entity(p1), is_health_care_provider(p2),
    is_for_treatment(u), is_phi(t), msg_contains(m, q, t).
// Exception (b)(2)(ii): disclosures to the individual
excluded_164_502_b_2(ARGS) :-
    is_covered_entity(p1),
    (p2 = q ; is_personal_representative(p2, q)),
    known_purpose(u), is_phi(t), msg_contains(m, q, t).
```

Listing 6: Exception-as-separate-path pattern (from hipaa_164_502.dl)

Pattern 3: Exception-as-disjunction. Multiple exception conditions are combined with ; (OR) directly in the rule body. This is used for simple, inline alternatives that do not warrant separate predicates.

Example — the "$p_2 \approx q$" pattern from the PDF, meaning "the receiver is either the individual or their personal representative":

```
permitted_by_164_502_a_1_i(ARGS, ...) :-
    is_covered_entity(p1),
    is_phi(t), msg_contains(m, q, t),
    known_purpose(u),
    (p2 = q ; is_personal_representative(p2, q)).  // inline disjunction
```

Listing 7: Exception-as-disjunction pattern (from hipaa_164_502.dl)

Design Decision

Choosing the right exception pattern depends on complexity. Single-condition exceptions use Pattern 3 (inline disjunction). Multi-condition exceptions that collect several cases use Pattern 1 (negation with a collector predicate). Exceptions that provide alternative satisfaction of a requirement use Pattern 2 (separate path).

4.4 Oracle Predicates vs. Derived Predicates

A fundamental distinction in the formalization is between facts that must be asserted externally and facts that are computed from rules.

Oracle predicates (EDB — Extensional Database). These are declared with `.decl` but populated only as ground facts, never derived by rules. They exist because HIPAA leaves certain determinations to human judgment, professional discretion, or external legal processes. Examples from `hipaa_macros.dl`:

```
// Professional judgment: entity believes disclosure is minimum necessary
.decl believes_minimum_necessary(DECL_ARGS)

// Contract-based: business associate has provided safeguard assurances
.decl satisfactory_assurances(p1:Principal, p2:Principal,
    q:Principal, t:Attribute, u:Purpose)

// Subjective belief: workforce member believes entity engaged in
// unlawful conduct (for whistleblower protection, section 502(j)(1))
.decl believes_unlawful_conduct(p1:Principal, ce:Principal)

// Legal determination: P has authority to act on behalf of Q
.decl has_authority_to_act(p:Principal, q:Principal)

// External determination: abuse exception for personal representative
.decl abuse_exception(p:Principal, q:Principal)
```

Listing 8: Oracle predicates (from hipaa_macros.dl and hipaa_164_502.dl)

Derived predicates (IDB — Intensional Database). These are computed from rules using hierarchy traversals, role checks, and rule composition. They have both a `.decl` and one or more rules with `:-`. Examples:

```
// Role check: derived from hierarchy
.decl is_covered_entity(p: Principal)
is_covered_entity(P) :- has_role(P, "covered-entity").
```

```
4
5  // Purpose check: derived from hierarchy
6  .decl is_for_tpo(u: Purpose)
7  is_for_tpo(U) :- is_for_treatment(U).
8  is_for_tpo(U) :- is_for_payment(U).
9  is_for_tpo(U) :- is_for_hc_ops(U).
10
11 // Personal representative: derived from authority + group membership + negation
12 is_personal_representative(P, Q) :-
13     has_authority_to_act(P, Q),
14     (belongs_to(Q, "adult") ; belongs_to(Q, "emancipated-minor")),
15     !abuse_exception(P, Q).
```

Listing 9: Derived predicates (from hipaa_macros.dl)

When to use which. Use oracle predicates for:

- Subjective judgments ("believes in good faith," "reasonably believes")
- Professional discretion ("minimum necessary in the professional judgment of")
- External legal determinations ("has legal authority," "is executor of estate")
- Contractual states ("obtained consent," "satisfactory assurances provided")

Use derived predicates for:

- Role checking via hierarchy (`is_covered_entity`, `is_health_care_provider`)
- Purpose classification (`is_for_tpo`, `is_for_treatment`)
- Attribute classification (`is_phi`, `is_dii`)
- Rule composition (any `permitted_by_` or `required_by_` predicate)

4.5 The Action Parameter Translation

The PDF formalization bundles all disclosure parameters into a single action variable A in Prolog style: `permitted_by_164_502(A)`. This works in Prolog because A can be a compound term (e.g., `action(p1, p2, q, m, t, u)`). Soufflé does not support compound terms as relation arguments, so we must **explicitly unpack** the action into six separate parameters:

PDF (Prolog-style)	$\longrightarrow$	**Soufflé**
`permitted_by_164_502(A)`		`permitted_by_164_502(p1, p2, q, m, t, u, E)`

To reduce verbosity, two C preprocessor macros are defined in `hipaa_types.dl`:

```
1  #define ARGS p1, p2, q, m, t, u
2  #define DECL_ARGS p1:Principal, p2:Principal, q:Principal, m:Message, t:Attribute, u:Purpose
```

Listing 10: Action parameter macros (from hipaa_types.dl)

`DECL_ARGS` is used in `.decl` declarations (where each variable needs a type annotation). `ARGS` is used in rule heads and bodies (where variables are already in scope). Every `predicate(A)` in the PDF becomes `predicate(ARGS)` in Soufflé, plus the explanation parameter `e: Expl` where applicable.

Warning

The macros expand to *variable names*, not fresh symbols. All rules in the same clause share the same variable namespace: `p1` in the head is the same `p1` in the body. This is essential for correct join semantics — the sender in the head must be the same sender checked by `is_covered_entity(p1)`.

4.6 Closed-World Assumption for Denial

Soufflé uses the **closed-world assumption**: if a tuple cannot be derived from the rules and facts, it is considered false. This has a profound implication for HIPAA compliance checking — *denial does not require explicit denial rules*. The absence of any permission pathway *is* the denial.

The top-level denial rule exploits this directly:

```
.decl disclosure_attempted(DECL_ARGS)
.decl is_disclosure_denied(DECL_ARGS)

// A disclosure is denied if it was attempted but not allowed
is_disclosure_denied(ARGS) :-
    disclosure_attempted(ARGS),
    !is_disclosure_allowed(ARGS, _).
```

Listing 11: Denial via closed-world assumption (from hipaa_top.dl)

The `disclosure_attempted` relation serves as the **universe of queries** — it defines the set of disclosures we want to check. For any attempted disclosure, if no rule chain can derive `is_disclosure_allowed` for that tuple, the negation succeeds and the disclosure is marked as denied. This mirrors the HIPAA Privacy Rule's fundamental principle: "A covered entity may not use or disclose protected health information, *except* as permitted or required." The default is prohibition.

Design Decision

This design means we never need to write rules of the form "deny if condition X holds." Instead, we only write positive permission rules. Any disclosure that fails to match a permission pathway is automatically denied. This dramatically simplifies the formalization and eliminates the risk of conflicting allow/deny rules.

4.7 Stub Pattern for Incremental Formalization

When building the formalization incrementally, not all referenced sections are formalized immediately. The **stub pattern** provides a safe default: declare the relation with `.decl` but provide no rules, yielding an empty relation.

```
// Positive stub: no disclosures permitted through this path
.decl permitted_by_164_512(DECL_ARGS, e: Expl)

// Negative stub: used in negation by other rules
.decl require_authorization_by_164_508(DECL_ARGS)
```

Listing 12: Stub declarations (from hipaa_stubs.dl)

The safety properties of stubs differ depending on how they are referenced:

Positive stubs (e.g., `permitted_by_164_512`): Being empty means no disclosures are permitted through that path. This is **conservative/safe** — we err on the side of denying disclosures until the section is formalized. For example, `permitted_by_164_502_a_1_vi` requires `permitted_by_164_512(ARGS, E)` in its body. Since the stub is empty, this rule never fires, blocking the §164.512 permission pathway.

Negative stubs (e.g., `require_authorization_by_164_508`): Being empty means the negation `!require_authorization_by_164_508(ARGS)` is always true. This is **permissive** — it means the "except when authorization is required" guard never blocks. In the case of §164.506(a), this means TPO disclosures are permitted even when an authorization *should* be required (e.g., for psychotherapy notes under §164.508(a)(2)).

Warning

Negative stubs create a permissiveness tradeoff. Until §164.508 is formalized, the system may allow disclosures that should require explicit patient authorization. When formalizing a new section, audit all stubs that are referenced in negation to determine whether the empty default is acceptable. The file `hipaa_stubs.dl` documents which sections reference each stub for exactly this purpose.

5 Type System and ADT Explanation Trees

5.1 The 6-Tuple Action Signature

The PDF formalization bundles a disclosure action as a single parameter A. We unpack it into a 6-tuple:

Variable	Type	Meaning
`p1`	`Principal`	Sender (the covered entity or disclosing party)
`p2`	`Principal`	Receiver (who receives the PHI)
`q`	`Principal`	Subject (the individual the PHI is about)
`m`	`Message`	Message identifier (which message is being disclosed)
`t`	`Attribute`	Attribute type (what kind of information: diagnosis, phi, dii, etc.)
`u`	`Purpose`	Purpose of the disclosure (treatment, payment, marketing, etc.)

Table 2: The 6-tuple action signature.

5.2 Preprocessor Macros for the Signature

To avoid repeating the 6-tuple everywhere, we define two C preprocessor macros:

```
#define ARGS p1, p2, q, m, t, u
#define DECL_ARGS p1:Principal, p2:Principal, q:Principal, m:Message, t:Attribute, u:Purpose
```

Listing 13: Preprocessor macros from hipaa_types.dl

`ARGS` is used in rule bodies and heads where the variables are already in scope. `DECL_ARGS` is used in `.decl` declarations where each variable needs its type annotation.

Example usage:

```
// Declaration uses DECL_ARGS
.decl permitted_by_164_506(DECL_ARGS, e: Expl)

// Rule head and body use ARGS
permitted_by_164_506(ARGS, E) :- permitted_by_164_506_a(ARGS, E).
```

After preprocessing, the declaration expands to:

```
.decl permitted_by_164_506(p1:Principal, p2:Principal, q:Principal,
    m:Message, t:Attribute, u:Purpose, e: Expl)
```

5.3 Primitive Domain Types

All types are declared in `hipaa_types.dl`:

```
.type Principal <: symbol   // people, organizations, entities
.type Role <: symbol        // covered-entity, provider, patient, etc.
.type Attribute <: symbol   // phi, dii, psychotherapy-notes, etc.
.type Purpose <: symbol     // treatment, payment, marketing, etc.
.type Message <: symbol     // message identifiers
.type TimePoint <: number   // discrete time points (for future use)
.type Rel <: symbol         // relationship identifiers
```

Listing 14: Type declarations from hipaa_types.dl

All are subtypes of `symbol` (strings) except `TimePoint` which is a subtype of `number`. The `<:` operator in Soufflé creates a named subtype, giving us type safety: you cannot accidentally pass a `Principal` where a `Purpose` is expected.

5.4 The `Expl` ADT: Explanation Trees

The most important design decision is the use of an Algebraic Data Type (ADT) to carry structured explanations through every derivation. The `Expl` type is a tagged union with four variants:

```
.type Expl = Leaf  { reason: symbol }
           | Step1 { reason: symbol, sub1: Expl }
           | Step2 { reason: symbol, sub1: Expl, sub2: Expl }
           | Step3 { reason: symbol, sub1: Expl, sub2: Expl, sub3: Expl }
```

Listing 15: The Expl ADT from hipaa_types.dl

`Leaf`: A base fact or oracle predicate. No sub-explanations. The `reason` string identifies the HIPAA section.

`Step1`: A rule with one sub-explanation. Used when a rule delegates to one child section.

`Step2`: A rule combining two sub-explanations. Used when both a positive and negative norm contribute.

`Step3`: A rule combining three sub-explanations.

ADT values are constructed with the `$` prefix:

```
$Leaf("164.506(c)(1): covered entity's own TPO")
$Step1("164.502(a)(1)(ii): TPO per 164.506", E)
```

5.5 How Explanation Trees Compose

When a disclosure is allowed, the explanation tree shows the full chain of reasoning. For example, consider a hospital sending blood test results to a doctor for treatment. The explanation tree would be:

```
Step1("164.502(a): permitted use by covered entity",
  Step1("164.502(a)(1)(ii): TPO per 164.506",
    Step1("164.506(a): standard TPO use/disclosure",
      Leaf("164.506(c)(2): treatment activities of health care provider")
    )
  )
)
```

Reading from the outside in:

1. §164.502(a) permits the use because the sender is a covered entity and the info is PHI.
2. Within §164.502(a), sub-section (a)(1)(ii) applies: this is a TPO disclosure, delegated to §164.506.
3. Within §164.506, sub-section (a) applies: no authorization required, the entity is covered, the purpose is TPO, and (c) is satisfied.
4. Within §164.506(c), sub-section (c)(2) applies: the receiver is a health care provider and the purpose is treatment.

This is far more informative than a simple "allowed/denied" answer. It provides a *legal citation trail* that can be audited.

6 Hierarchy Design

HIPAA rules frequently refer to categories of entities, information, and purposes. Rather than enumerating every possible value, we define hierarchies with transitive closure. This means that if `doctor` is a subtype of `provider`, and a rule requires the receiver to be a `provider`, then a `doctor` automatically qualifies.

6.1 Role Hierarchy

The role hierarchy captures specialization relationships among HIPAA principals. It is defined with `role_isa(child, parent)` facts:

```
// Health care providers
role_isa("psychiatrist", "doctor").
role_isa("doctor", "provider").
role_isa("nurse", "provider").
role_isa("pharmacist", "provider").
role_isa("lab-technician", "provider").
role_isa("provider", "covered-entity").

// Health plans
role_isa("health-insurance-issuer", "health-plan").
role_isa("HMO", "health-plan").
role_isa("group-health-plan", "health-plan").
role_isa("health-plan", "covered-entity").
```

```
14
15 // Health care clearinghouses
16 role_isa("clearinghouse", "covered-entity").
17
18 // Hospitals
19 role_isa("hospital", "covered-entity").
20
21 // Business associates
22 role_isa("billing-company", "business-associate").
23 role_isa("cloud-storage", "business-associate").
24 role_isa("transcription-service", "business-associate").
25
26 // Oversight and government
27 role_isa("oversight-agency", "government-entity").
28 role_isa("public-health-authority", "government-entity").
29 role_isa("law-enforcement-official", "government-entity").
30
31 // Guardian types (for 164.502(g))
32 role_isa("parent", "guardian-type").
33 role_isa("legal-guardian", "guardian-type").
34 role_isa("loco-parentis", "guardian-type").
```

Listing 16: Role hierarchy from hipaa_hierarchies.dl

The full hierarchy tree:

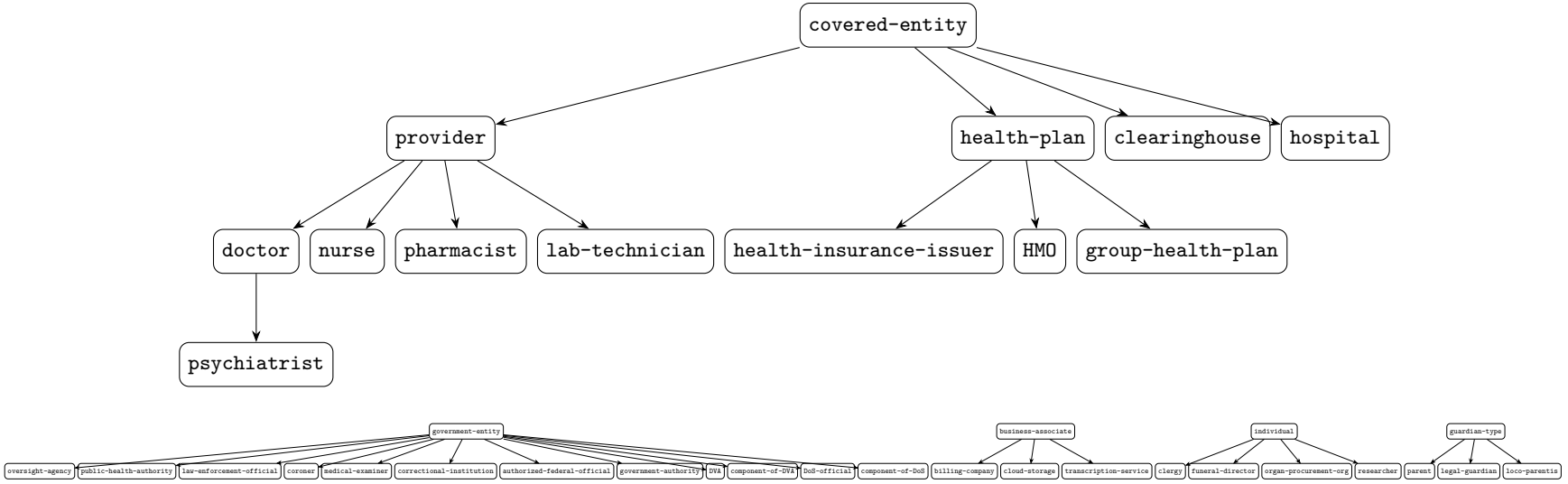

Figure 3: Complete role hierarchy. Arrows point from child to parent (specialization). The hierarchy now includes government entities, business associates, specialized individuals, and guardian types added for §§508–524.

6.2 Transitive Closure

The transitive closure is computed recursively:

```
.decl role_subtype(child: Role, ancestor: Role)
role_subtype(C, P) :- role_isa(C, P).
role_subtype(C, A) :- role_isa(C, P), role_subtype(P, A).
```

Listing 17: Transitive closure for roles

This means that `role_subtype("psychiatrist", "covered-entity")` holds (via psychiatrist < doctor < provider < covered-entity).

The `has_role` predicate checks both direct assignment and hierarchy:

```
.decl activerole(p: Principal, r: Role)

.decl has_role(p: Principal, r: Role)
has_role(P, R) :- activerole(P, R).
has_role(P, R) :- activerole(P, R2), role_subtype(R2, R).
```

Listing 18: has_role uses both direct and inherited roles

So if `activerole("dr_smith", "doctor")` is a fact, then `has_role("dr_smith", "provider")` and `has_role("dr_smith", "covered-entity")` are both derived.

6.3 Attribute Hierarchy

The attribute hierarchy classifies types of health information:

```
// PHI subtypes
attr_isa("psychotherapy-notes", "phi").
attr_isa("genetic-info", "phi").
attr_isa("diagnosis", "phi").
attr_isa("prescription", "phi").
attr_isa("medical-record", "phi").
attr_isa("billing-record", "phi").
attr_isa("blood-test-results", "phi").
attr_isa("lab-results", "phi").
attr_isa("treatment-plan", "phi").
attr_isa("patient-name", "phi").
attr_isa("patient-location", "phi").
attr_isa("patient-condition", "phi").
attr_isa("religious-affiliation", "phi").

// De-identified information (NOT phi)
attr_isa("statistical-data", "dii").
attr_isa("anonymized-record", "dii").

// Directory information (164.510)
attr_isa("patient-name", "directory-information").
attr_isa("patient-location", "directory-information").
```

Listing 19: Attribute hierarchy (selected entries)

Note that some attributes belong to multiple hierarchies: `patient-name` is both a subtype of `phi` and `directory-information`. This is intentional — HIPAA treats directory information as a special category of PHI with its own rules (§164.510).

The transitive closure follows the same pattern:

```
.decl attr_subtype(child: Attribute, ancestor: Attribute)
attr_subtype(C, P) :- attr_isa(C, P).
attr_subtype(C, A) :- attr_isa(C, P), attr_subtype(P, A).
```

6.4 Purpose Hierarchy

The purpose hierarchy classifies the reasons for disclosure:

```
// Treatment sub-purposes
purpose_isa("surgery", "treatment").
purpose_isa("referral", "treatment").
purpose_isa("consultation", "treatment").
purpose_isa("emergency-treatment", "treatment").

// Payment sub-purposes
purpose_isa("billing", "payment").
purpose_isa("claims-processing", "payment").
purpose_isa("eligibility-determination", "payment").
purpose_isa("reimbursement", "payment").

// Healthcare operations sub-purposes
purpose_isa("quality-assessment", "healthcare-operations").
purpose_isa("quality-improvement", "healthcare-operations").
purpose_isa("case-management", "healthcare-operations").
purpose_isa("care-coordination", "healthcare-operations").
purpose_isa("competency-assurance", "healthcare-operations").
purpose_isa("fraud-detection", "healthcare-operations").
purpose_isa("compliance-audit", "healthcare-operations").
purpose_isa("business-planning", "healthcare-operations").
purpose_isa("accreditation", "healthcare-operations").
purpose_isa("training-programs", "healthcare-operations").
```

Listing 20: Purpose hierarchy (selected entries)

The purpose hierarchy also includes a `known_purpose` relation that collects all purpose values. This is needed to ground unbound purpose variables (see Section **??**).

```
.decl known_purpose(u: Purpose)
known_purpose(U) :- purpose_isa(U, _).
known_purpose(U) :- purpose_isa(_, U).
known_purpose("treatment").
known_purpose("payment").
known_purpose("healthcare-operations").
known_purpose("marketing").
known_purpose("research").
// ... and other top-level purposes
```

Listing 21: known_purpose for grounding

7 Macro/Helper Predicates

The PDF formalization uses macros like `is_from_coveredEntity(A)`, `is_phi(A)`, `is_for_treatment(A)`, and `is_for_eitherPurpose(A)`. These are translated to Soufflé helper predicates.

7.1 Entity Role Checks

The PDF macro `is_from_coveredEntity(A)` checks that the sender (p1) has the role "covered-entity." In Soufflé:

```
.decl is_covered_entity(p: Principal)
is_covered_entity(P) :- has_role(P, "covered-entity").

.decl is_health_care_provider(p: Principal)
is_health_care_provider(P) :- has_role(P, "provider").

.decl is_health_plan(p: Principal)
is_health_plan(P) :- has_role(P, "health-plan").

.decl is_business_associate(p: Principal)
is_business_associate(P) :- has_role(P, "business-associate").

.decl is_secretary(p: Principal)
is_secretary(P) :- activerole(P, "secretary").
```

Listing 22: Entity role check predicates

Note that `is_secretary` uses `activerole` directly (not `has_role`) because the HHS Secretary is a unique role with no subtypes.

7.2 Attribute and Purpose Checks

```
// is_phi(T): T is protected health information (or a subtype)
.decl is_phi(t: Attribute)
is_phi("phi").
is_phi(T) :- attr_subtype(T, "phi").

// is_dii(T): T is de-identified information
.decl is_dii(t: Attribute)
is_dii("dii").
is_dii(T) :- attr_subtype(T, "dii").

// is_for_tpo(U): purpose is treatment, payment, or healthcare operations
// This is the PDF's is_for_eitherPurpose macro
.decl is_for_tpo(u: Purpose)
is_for_tpo(U) :- is_for_treatment(U).
is_for_tpo(U) :- is_for_payment(U).
is_for_tpo(U) :- is_for_hc_ops(U).
```

Listing 23: Attribute and purpose check predicates

7.3 Personal Representative (§164.502(g))

The personal representative macro is one of the most complex translations. In HIPAA, a personal representative may act on behalf of an individual. The rules differ by the subject's status (adult, emancipated minor, unemancipated minor, deceased).

```
// 502(g)(2): Adults and emancipated minors
is_personal_representative(P, Q) :-
    has_authority_to_act(P, Q),
    (belongs_to(Q, "adult") ; belongs_to(Q, "emancipated-minor")),
    !abuse_exception(P, Q).
```

```
6
7  // 502(g)(3): Unemancipated minors with guardian
8  is_personal_representative(P, Q) :-
9      is_guardian(P, Q),
10     belongs_to(Q, "unemancipated-minor"),
11     has_authority_to_act(P, Q),
12     !minor_acts_as_individual(P, Q),
13     !abuse_exception(P, Q).
14
15 // 502(g)(4): Deceased individuals
16 is_personal_representative(P, Q) :-
17     (activerole(P, "executor") ; activerole(P, "administrator")),
18     belongs_to(Q, "deceased"),
19     has_authority_to_act(P, Q),
20     !abuse_exception(P, Q).
```

Listing 24: Personal representative rules from hipaa_macros.dl

Key design points:

- `has_authority_to_act(P, Q)` is an oracle predicate — it must be asserted as a fact.
- `abuse_exception(P, Q)` is also an oracle. When asserted, it blocks personal representative status.
- `minor_acts_as_individual(G, M)` is an oracle for cases where the minor acts on their own behalf (e.g., minor consented to treatment independently).
- The disjunction `(belongs_to(Q, "adult") ; belongs_to(Q, "emancipated-minor"))` mirrors the PDF's union of categories.

7.4 Oracle Predicates

Oracle predicates represent conditions that cannot be determined from the fact database alone — they require external judgment or real-world verification. Examples:

```
1  // Does the CE believe the disclosure is minimum necessary?
2  .decl believes_minimum_necessary(p1:Principal, p2:Principal,
3      q:Principal, m:Message, t:Attribute, u:Purpose)
4
5  // Was consent obtained for TPO under 164.506(b)?
6  .decl obtained_consent_506b(p1:Principal, p2:Principal,
7      q:Principal, t:Attribute, u:Purpose)
8
9  // Has the BA provided satisfactory assurances?
10 .decl satisfactory_assurances(p1:Principal, p2:Principal,
11     q:Principal, t:Attribute, u:Purpose)
12
13 // Good-faith belief of unlawful conduct (whistleblower)
14 .decl believes_unlawful_conduct(employee:Principal, employer:Principal)
```

Listing 25: Oracle predicates from hipaa_macros.dl

These are declared with `.decl` but have no rules — they must be populated as facts in the fact database.

8 Formalization of §164.506

§164.506 governs uses and disclosures for Treatment, Payment, or Health Care Operations (TPO). It is structured as a disjunction of three sub-sections.

8.1 §164.506 — Top-Level

HIPAA Regulation Text

§164.506 permits disclosures under sub-sections (a), (b), or (c).

PDF Formal Notation

```
permitted_by_164_506(A) :- permitted_by_164_506_a(A) ;
permitted_by_164_506_b(A) ; permitted_by_164_506_c(A).
```

```
.decl permitted_by_164_506(DECL_ARGS, e: Expl)

permitted_by_164_506(ARGS, E) :- permitted_by_164_506_a(ARGS, E).
permitted_by_164_506(ARGS, E) :- permitted_by_164_506_b(ARGS, E).
permitted_by_164_506(ARGS, E) :- permitted_by_164_506_c(ARGS, E).
```

Listing 26: §164.506 top-level

The disjunction in the PDF (;) becomes three separate rules in Soufflé, each contributing to the same head relation. This is idiomatic Soufflé: multiple rules for the same relation are implicitly OR'd.

8.2 §164.506(a) — Standard Permitted Uses

HIPAA Regulation Text

"Except with respect to uses or disclosures that require an authorization under sections 164.508(a)(2) and (3), a covered entity may use or disclose protected health information for treatment, payment, or health care operations as set forth in paragraph (c) of this section, provided that such use or disclosure is consistent with other applicable requirements of this subpart."

PDF Formal Notation

```
permitted_by_164_506_a(A) :-
\+ require_authorization_by_164_508(A),
is_from_coveredEntity(A), is_for_eitherPurpose(A),
permitted_by_164_506_c(A),
writeln('HIPAA rule 164_506_a;').
```

```
.decl permitted_by_164_506_a(DECL_ARGS, e: Expl)

permitted_by_164_506_a(ARGS,
    $Step1("164.506(a): standard TPO use/disclosure
            (no authorization required)", E)) :-
```

```
    !require_authorization_by_164_508(ARGS),
    is_covered_entity(p1),
    is_phi(t),
    msg_contains(m, q, t),
    is_for_tpo(u),
    permitted_by_164_506_c(ARGS, E).
```

Listing 27: §164.506(a) — Soufflé translation

Design Decision

The PDF's `\+ require_authorization_by_164_508(A)` (Prolog negation-as-failure) becomes `!require_authorization_by_164_508(ARGS)` in Soufflé. Since §164.508 is a stub (no rules, no tuples), this negation always succeeds — effectively making the "except" clause vacuous until §164.508 is formalized. This is the correct behavior: we are saying "the authorization exception does not apply" because we have no evidence that it does.

Key observations:

- The PDF's `is_from_coveredEntity(A)` is expanded to `is_covered_entity(p1)`.
- The PDF's `is_for_eitherPurpose(A)` is expanded to `is_for_tpo(u)`.
- We add `is_phi(t)` and `msg_contains(m, q, t)` to ensure the message actually contains PHI about the subject.
- The PDF's `writeln` (debug output) is replaced by the ADT explanation string in the head.
- The delegation to `permitted_by_164_506_c(ARGS, E)` threads the sub-explanation through.

8.3 §164.506(b) — Consent for TPO

HIPAA Regulation Text

"A covered entity may obtain consent of the individual to use or disclose protected health information to carry out treatment, payment, or health care operations."

PDF Formal Notation

```
permitted_by_164_506_b(A) :- is_for_eitherPurpose(A),
is_consentedby_about(A).
```

```
.decl permitted_by_164_506_b(DECL_ARGS, e: Expl)

permitted_by_164_506_b(ARGS,
    $Leaf("164.506(b)(1): consent obtained for TPO")) :-
    is_covered_entity(p1),
    is_phi(t),
    msg_contains(m, q, t),
    is_for_tpo(u),
    obtained_consent_506b(p1, p2, q, t, u).
```

Listing 28: §164.506(b) — Consent for TPO

This uses a `$Leaf` explanation because consent is a terminal fact — it does not delegate further.

8.4 §164.506(c) — Implementation Specification

HIPAA Regulation Text

The implementation specification is a disjunction of five sub-rules (c)(1) through (c)(5), each covering a different TPO scenario.

```
.decl permitted_by_164_506_c(DECL_ARGS, e: Expl)

permitted_by_164_506_c(ARGS, E) :- permitted_by_164_506_c_1(ARGS, E).
permitted_by_164_506_c(ARGS, E) :- permitted_by_164_506_c_2(ARGS, E).
permitted_by_164_506_c(ARGS, E) :- permitted_by_164_506_c_3(ARGS, E).
permitted_by_164_506_c(ARGS, E) :- permitted_by_164_506_c_4(ARGS, E).
permitted_by_164_506_c(ARGS, E) :- permitted_by_164_506_c_5(ARGS, E).
```

Listing 29: §164.506(c) — disjunction of sub-rules

8.4.1 §164.506(c)(1) — Own TPO

HIPAA Regulation Text

"A covered entity may use or disclose protected health information for its own treatment, payment, or health care operations."

```
permitted_by_164_506_c_1(ARGS,
    $Leaf("164.506(c)(1): covered entity's own TPO")) :-
    is_covered_entity(p1),
    is_phi(t),
    msg_contains(m, q, t),
    is_for_tpo(u),
    // "its own" -- the message stays within the organization
    (organization_member(p2, p1) ; p1 = p2).
```

Listing 30: §164.506(c)(1)

Design Decision

"Its own" TPO means the disclosure stays within the covered entity's organization. We model this with the disjunction: either `p2` is a member of `p1`'s organization, or `p1` is using it internally (`p1 = p2`).

8.4.2 §164.506(c)(2) — Treatment Activities of a Provider

HIPAA Regulation Text

"A covered entity may disclose protected health information for treatment activities of a health care provider."

```
permitted_by_164_506_c_2(ARGS,
    $Leaf("164.506(c)(2): treatment activities of
           health care provider")) :-
    is_covered_entity(p1),
    is_health_care_provider(p2),
```

```
    is_phi(t),
    msg_contains(m, q, t),
    is_for_treatment(u).
```

Listing 31: §164.506(c)(2)

This is the rule that permits, for example, a hospital to send blood test results to a doctor for treatment purposes. The receiver must be a health care provider (doctor, nurse, pharmacist, etc.) and the purpose must be treatment.

8.4.3 §164.506(c)(3) — Payment Activities of Receiving Entity

HIPAA Regulation Text

"A covered entity may disclose protected health information to another covered entity or a health care provider for the payment activities of the entity that receives the information."

```
permitted_by_164_506_c_3(ARGS,
    $Leaf("164.506(c)(3): payment activities of
           receiving entity")) :-
    is_covered_entity(p1),
    (is_covered_entity(p2) ; is_health_care_provider(p2)),
    is_phi(t),
    msg_contains(m, q, t),
    is_for_payment(u).
```

Listing 32: §164.506(c)(3)

The receiver can be either a covered entity or a health care provider. The purpose must be payment (or a sub-purpose like billing, claims-processing, etc.).

8.4.4 §164.506(c)(4) — Healthcare Operations Between Covered Entities

HIPAA Regulation Text

"A covered entity may disclose protected health information to another covered entity for health care operations activities of the entity that receives the information, if each entity either has or had a relationship with the individual who is the subject of the protected health information being requested, the protected health information pertains to such relationship, and the disclosure is [for quality assessment, fraud detection, or compliance]."

```
permitted_by_164_506_c_4(ARGS,
    $Leaf("164.506(c)(4): healthcare ops between
           related covered entities")) :-
    is_covered_entity(p1),
    is_covered_entity(p2),
    p1 != p2,
    is_phi(t),
    msg_contains(m, q, t),
    // Both entities have/had relationship with subject
    inrelationship(p1, R1, q),
    pertains_to(t, R1),
    inrelationship(p2, R2, q),
    pertains_to(t, R2),
```

```
    // Purpose limited to specific healthcare operations
    (is_for_hc_ops_paras_1_2(u) ;
     is_for_fraud_detection(u) ;
     is_for_compliance(u)).
```

Listing 33: §164.506(c)(4)

This is the most complex sub-rule. It requires:

1. Both sender and receiver are covered entities (and different).
2. Both have (or had) a relationship with the subject individual.
3. The PHI pertains to those relationships.
4. The purpose is limited to specific healthcare operations (paragraphs (1) and (2) of the definition), fraud detection, or compliance.

Design Decision

The PDF uses temporal logic ("has or had a relationship," written as $\Diamond$`inrelationship`(p_1, r_1, q)) to express that the relationship existed at some past time. Since we defer temporal translation for these sections (see Section **??**), we use the atemporal `inrelationship` predicate directly.

8.4.5 §164.506(c)(5) — Organized Health Care Arrangement

HIPAA Regulation Text

"A covered entity that participates in an organized health care arrangement may disclose protected health information about an individual to another covered entity that participates in the organized health care arrangement for any health care operations activities of the organized health care arrangement."

```
permitted_by_164_506_c_5(ARGS,
    $Leaf("164.506(c)(5): organized health care
           arrangement")) :-
    is_covered_entity(p1),
    is_covered_entity(p2),
    is_phi(t),
    msg_contains(m, q, t),
    is_for_hc_ops(u),
    // Both participate in the same OHCA
    participates_in_ohca(p1, Arr),
    participates_in_ohca(p2, Arr).
```

Listing 34: §164.506(c)(5)

Both entities must participate in the same organized health care arrangement (OHCA), and the purpose must be healthcare operations of that arrangement.

9 Formalization of §164.502

§164.502 is the "General Rules" section — the gatekeeper for all HIPAA disclosures. It is structured as a disjunction of sub-sections (a) through (j), though several are constraints or special cases rather than independent permission pathways.

9.1 §164.502 — Top-Level

```
.decl permitted_by_164_502(DECL_ARGS, e: Expl)

permitted_by_164_502(ARGS, E) :- permitted_by_164_502_a(ARGS, E).
// (b): minimum necessary is a CONSTRAINT, not independent permission
permitted_by_164_502(ARGS, E) :- permitted_by_164_502_c(ARGS, E).
permitted_by_164_502(ARGS, E) :- permitted_by_164_502_d(ARGS, E).
permitted_by_164_502(ARGS, E) :- permitted_by_164_502_e(ARGS, E).
// (f): deceased treated same as living -- no rule
// (g): personal representatives -- handled in macros
permitted_by_164_502(ARGS, E) :- permitted_by_164_502_h(ARGS, E).
permitted_by_164_502(ARGS, E) :- permitted_by_164_502_i(ARGS, E).
permitted_by_164_502(ARGS, E) :- permitted_by_164_502_j(ARGS, E).
```

Listing 35: §164.502 top-level

Design Decision

Sub-section (b) (minimum necessary) is deliberately *not* an independent permission pathway. It is a constraint that modifies other pathways. Sub-section (f) (deceased individuals) requires no rule because deceased individuals are treated the same as living ones for PHI purposes. Sub-section (g) (personal representatives) is handled as a macro/helper predicate rather than a permission rule.

9.2 §164.502(a) — Permitted and Required Disclosures

HIPAA Regulation Text

"A covered entity may not use or disclose protected health information, except as permitted or required by [this subpart or subpart C of part 160]."

This is the gatekeeper: it requires `is_covered_entity(p1)`, `is_phi(t)`, and `msg_contains(m, q, t)`, then dispatches to either (a)(1) for permitted uses or (a)(2) for required disclosures.

```
.decl permitted_by_164_502_a(DECL_ARGS, e: Expl)

// Permitted use
permitted_by_164_502_a(ARGS,
    $Step1("164.502(a): permitted use by covered entity", E)) :-
    is_covered_entity(p1), is_phi(t), msg_contains(m, q, t),
    permitted_by_164_502_a_1(ARGS, E).

// Required disclosure
permitted_by_164_502_a(ARGS,
    $Step1("164.502(a): required disclosure by covered entity", E)) :-
    is_covered_entity(p1), is_phi(t), msg_contains(m, q, t),
    required_by_164_502_a_2(ARGS, E).
```

```
14
15 // Permitted under subpart C of part 160
16 permitted_by_164_502_a(ARGS,
17     $Leaf("164.502(a): permitted under 160.C")) :-
18     is_covered_entity(p1), is_phi(t), msg_contains(m, q, t),
19     permitted_by_160_C(ARGS).
```

Listing 36: §164.502(a)

9.3 §164.502(a)(1) — Permitted Uses (i through vi)

This is a disjunction of six permission pathways:

```
1  .decl permitted_by_164_502_a_1(DECL_ARGS, e: Expl)
2
3  permitted_by_164_502_a_1(ARGS, E) :-
4      permitted_by_164_502_a_1_i(ARGS, E).    // to the individual
5  permitted_by_164_502_a_1(ARGS, E) :-
6      permitted_by_164_502_a_1_ii(ARGS, E).   // TPO per 164.506
7  permitted_by_164_502_a_1(ARGS, E) :-
8      permitted_by_164_502_a_1_iii(ARGS, E).  // incident to use
9  permitted_by_164_502_a_1(ARGS, E) :-
10     permitted_by_164_502_a_1_iv(ARGS, E).   // authorization per 164.508
11 permitted_by_164_502_a_1(ARGS, E) :-
12     permitted_by_164_502_a_1_v(ARGS, E).    // agreement per 164.510
13 permitted_by_164_502_a_1(ARGS, E) :-
14     permitted_by_164_502_a_1_vi(ARGS, E).   // per 164.512/514
```

Listing 37: §164.502(a)(1) — six pathways

9.3.1 §164.502(a)(1)(i) — To the Individual

HIPAA Regulation Text

"To the individual."

PDF Formal Notation

$\varphi^{+}_{164.502a1i}$: $\texttt{activerole}(p_1, \text{covered-entity}) \wedge (p_2 \approx q) \wedge (t \in_T \text{phi})$
where $(p_2 \approx q) \triangleq \texttt{activerole}(p_2, \text{personal-representative}(q)) \vee (p_2 = q)$

```
1  permitted_by_164_502_a_1_i(ARGS,
2      $Leaf("164.502(a)(1)(i): disclosure to the individual
3             or personal representative")) :-
4      is_covered_entity(p1),
5      is_phi(t),
6      msg_contains(m, q, t),
7      known_purpose(u),
8      (p2 = q ; is_personal_representative(p2, q)).
```

Listing 38: §164.502(a)(1)(i) — disclosure to individual

Design Decision

The notation $p_2 \approx q$ from the PDF is translated as `(p2 = q ; is_personal_representative(p2, q))`. This captures the HIPAA principle that a personal representative stands in the shoes of the individual. The `known_purpose(u)` literal is needed for grounding — the purpose is unrestricted (any purpose is valid for disclosures to the individual), but every variable must be bound by some positive literal. See Section **??**.

9.3.2 §164.502(a)(1)(ii) — TPO per §164.506

HIPAA Regulation Text

"For treatment, payment, or health care operations, as permitted by and in compliance with §164.506."

```
permitted_by_164_502_a_1_ii(ARGS,
    $Step1("164.502(a)(1)(ii): TPO per 164.506", E)) :-
    is_for_tpo(u),
    permitted_by_164_506(ARGS, E).
```

Listing 39: §164.502(a)(1)(ii) — delegates to §164.506

This is the bridge between §164.502 and §164.506. The purpose must be TPO, and the actual permission check delegates to §164.506.

9.3.3 §164.502(a)(1)(iii) — Incident to Use

HIPAA Regulation Text

"Incident to a use or disclosure otherwise permitted or required by this subpart, provided that the covered entity has complied with the applicable requirements of §164.502(b), §164.514(d), and §164.530(c)."

```
.decl is_incident_to_use(DECL_ARGS)

permitted_by_164_502_a_1_iii(ARGS,
    $Leaf("164.502(a)(1)(iii): incident to otherwise
           permitted use/disclosure")) :-
    is_incident_to_use(ARGS),
    minimum_necessary_satisfied(ARGS),
    permitted_by_164_514_d(ARGS),
    permitted_by_164_530_c(ARGS).
```

Listing 40: §164.502(a)(1)(iii) — incident to use

"Incident to use" is an oracle predicate — it means the disclosure was incidental to a primary permitted use. The rule also requires compliance with minimum necessary (§502(b)), de-identification standards (§514(d)), and safeguards (§530(c)). Since §514(d) and §530(c) are stubs, this rule currently requires all three conditions but only `minimum_necessary_satisfied` can actually be satisfied.

9.3.4 §164.502(a)(1)(iv) — Valid Authorization per §164.508

HIPAA Regulation Text

"Pursuant to and in compliance with a valid authorization under §164.508."

```
permitted_by_164_502_a_1_iv(ARGS,
    $Leaf("164.502(a)(1)(iv): valid authorization
           per 164.508")) :-
    obtained_authorization_164_508(p1, p2, q, t, u),
    is_phi(t),
    msg_contains(m, q, t).
```

Listing 41: §164.502(a)(1)(iv) — authorization

9.3.5 §164.502(a)(1)(v) — Agreement per §164.510

```
permitted_by_164_502_a_1_v(ARGS,
    $Step1("164.502(a)(1)(v): agreement or permission
            per 164.510", E)) :-
    permitted_by_164_510(ARGS, E).
```

Listing 42: §164.502(a)(1)(v) — agreement

9.3.6 §164.502(a)(1)(vi) — Per §164.512, §164.514(e)(f)(g)

```
permitted_by_164_502_a_1_vi(ARGS,
    $Step1("164.502(a)(1)(vi): per 164.512", E)) :-
    permitted_by_164_512(ARGS, E).
permitted_by_164_502_a_1_vi(ARGS,
    $Step1("164.502(a)(1)(vi): per 164.514(e)", E)) :-
    permitted_by_164_514_e(ARGS, E).
// ... similarly for 164.514(f) and 164.514(g)
```

Listing 43: §164.502(a)(1)(vi) — other sections

9.4 §164.502(a)(2) — Required Disclosures

HIPAA has two cases where disclosure is *required* (not merely permitted):

9.4.1 §164.502(a)(2)(i) — To Individual Upon Request

HIPAA Regulation Text

"To an individual, when requested under §164.524 or §164.528."

```
required_by_164_502_a_2_i(ARGS,
    $Leaf("164.502(a)(2)(i): required disclosure to
           individual per 164.524/528")) :-
    (p2 = q ; is_personal_representative(p2, q)),
    is_reply_to_request(ARGS),
    (required_by_164_524(ARGS) ; required_by_164_528(ARGS)),
    is_phi(t),
    msg_contains(m, q, t).
```

Listing 44: §164.502(a)(2)(i)

9.4.2 §164.502(a)(2)(ii) — To the Secretary for Investigation

HIPAA Regulation Text

"To the Secretary under subpart C of part 160 of this subchapter, when the Secretary is undertaking an investigation or determination."

```
.decl secretary_investigation_authorized(DECL_ARGS)

required_by_164_502_a_2_ii(ARGS,
    $Leaf("164.502(a)(2)(ii): required disclosure to
           Secretary for investigation")) :-
    is_covered_entity(p1),
    is_secretary(p2),
    is_phi(t),
    msg_contains(m, q, t),
    is_for_investigation(u),
    secretary_investigation_authorized(ARGS).
```

Listing 45: §164.502(a)(2)(ii)

Design Decision

The PDF requires `permitted_by_160_C(A)`, but since that is a stub, including it would block this rule entirely. Instead, we use `secretary_investigation_authorized` as an oracle that substitutes until §160.C is formalized. The stubbed requirement is noted in a comment.

9.5 §164.502(b) — Minimum Necessary Standard

HIPAA Regulation Text

"When using or disclosing protected health information or when requesting protected health information from another covered entity or business associate, a covered entity or business associate must make reasonable efforts to limit protected health information to the minimum necessary to accomplish the intended purpose of the use, disclosure, or request."

Warning

This is a **CONSTRAINT**, not an independent permission pathway. It is enforced as a guard within other rules, not as a standalone rule that permits disclosures.

```
.decl minimum_necessary_satisfied(DECL_ARGS)

// (b)(1): entity believes minimum necessary
minimum_necessary_satisfied(ARGS) :-
    believes_minimum_necessary(ARGS).

// (b)(2): exceptions -- minimum necessary does not apply to:
```

```
minimum_necessary_satisfied(ARGS) :-
    excluded_164_502_b_2(ARGS).
```

Listing 46: §164.502(b) — minimum necessary

The minimum necessary standard is satisfied either if the entity believes it (oracle) or if one of the exceptions in (b)(2) applies.

The exceptions under §164.502(b)(2):

```
.decl excluded_164_502_b_2(DECL_ARGS)

// (b)(2)(i): disclosures to provider for treatment
excluded_164_502_b_2(ARGS) :-
    is_covered_entity(p1), is_health_care_provider(p2),
    is_for_treatment(u), is_phi(t), msg_contains(m, q, t).

// (b)(2)(ii): disclosures to the individual
excluded_164_502_b_2(ARGS) :-
    is_covered_entity(p1),
    (p2 = q ; is_personal_representative(p2, q)),
    known_purpose(u), is_phi(t), msg_contains(m, q, t).

// (b)(2)(iii): pursuant to authorization under 164.508
excluded_164_502_b_2(ARGS) :-
    obtained_authorization_164_508(p1, p2, q, t, u),
    is_phi(t), msg_contains(m, q, t).

// (b)(2)(iv): disclosures to Secretary per subpart C of 160
excluded_164_502_b_2(ARGS) :-
    is_secretary(p2), permitted_by_160_C(ARGS).

// (b)(2)(v): disclosures required by law (164.512(a))
excluded_164_502_b_2(ARGS) :-
    required_by_164_512_a(ARGS).

// (b)(2)(vi): required for compliance
excluded_164_502_b_2(ARGS) :-
    required_by_164_502_a_2(ARGS, _).
```

Listing 47: §164.502(b)(2) — minimum necessary exceptions

9.6 §164.502(d) — De-identified Information

HIPAA Regulation Text

§164.502(d)(1): A covered entity may use PHI to create de-identified information or disclose PHI to a business associate for such purpose.
§164.502(d)(2): De-identified information (meeting §164.514 standards) is not PHI and may be freely disclosed.

```
// (d)(1): Disclose to BA to create de-identified info
permitted_by_164_502_d(ARGS,
    $Leaf("164.502(d)(1): disclose to BA to create
           de-identified info")) :-
    is_covered_entity(p1),
```

```
    is_phi(t),
    msg_contains(m, q, t),
    is_business_associate_of(p2, p1),
    is_for_create_deidentified_info(u).

// (d)(2): De-identified info bypasses PHI rules
permitted_by_164_502_d(ARGS,
    $Leaf("164.502(d)(2): de-identified information
           is not PHI")) :-
    is_covered_entity(p1),
    is_dii(t),
    msg_contains(m, q, t),
    disclosure_attempted(ARGS).
```

Listing 48: §164.502(d) — de-identified info

Design Decision

Sub-section (d)(2) uses `is_dii(t)` instead of `is_phi(t)` because de-identified information is *not* PHI. This is the only rule that checks for `dii` instead of `phi`. The `disclosure_attempted(ARGS)` literal is needed to ground the purpose variable `u` (see Section **??**).

9.7 §164.502(e) — Business Associates

HIPAA Regulation Text

"A covered entity may disclose protected health information to a business associate and may allow a business associate to create or receive protected health information on its behalf, if the covered entity obtains satisfactory assurance that the business associate will appropriately safeguard the information."

```
// CE -> BA disclosure
permitted_by_164_502_e(ARGS,
    $Leaf("164.502(e)(1)(i): disclosure to business associate
           with safeguard assurances")) :-
    is_covered_entity(p1),
    is_business_associate_of(p2, p1),
    is_phi(t),
    msg_contains(m, q, t),
    is_for_ba_purpose(u),
    satisfactory_assurances(p1, p2, q, t, u),
    !excluded_164_502_e_1_ii(ARGS).

// BA -> CE disclosure
permitted_by_164_502_e(ARGS,
    $Leaf("164.502(e)(1)(i): disclosure from business associate
           to covered entity")) :-
    is_covered_entity(p2),
    is_business_associate_of(p1, p2),
    is_phi(t),
    msg_contains(m, q, t),
    is_for_ba_purpose(u),
    satisfactory_assurances(p2, p1, q, t, u),
    !excluded_164_502_e_1_ii(ARGS).
```

Listing 49: §164.502(e) — business associates

The exclusions under §164.502(e)(1)(ii) remove certain disclosures from the business associate requirement:

```
.decl excluded_164_502_e_1_ii(DECL_ARGS)

// (A) CE to provider for treatment
excluded_164_502_e_1_ii(ARGS) :-
    is_covered_entity(p1), is_health_care_provider(p2),
    is_for_treatment(u), is_phi(t), msg_contains(m, q, t).

// (B) Group health plan disclosures per 164.504(f)
excluded_164_502_e_1_ii(ARGS) :-
    (is_health_plan(p1) ; has_role(p1, "health-insurance-issuer")),
    permitted_by_164_504_f(ARGS),
    is_phi(t), msg_contains(m, q, t).

// (C) Government program providing public benefits
excluded_164_502_e_1_ii(ARGS) :-
    has_role(p1, "government-entity"), is_health_plan(p2),
    known_purpose(u), is_phi(t), msg_contains(m, q, t).
```

Listing 50: §164.502(e)(1)(ii) — exclusions

9.8 §164.502(h) — Provider/Plan Communication

```
permitted_by_164_502_h(ARGS,
    $Leaf("164.502(h): provider/plan communication
           per 164.522(b)")) :-
    (is_health_care_provider(p1) ; is_health_plan(p1)),
    is_phi(t), msg_contains(m, q, t),
    permitted_by_164_522_b(ARGS).
```

Listing 51: §164.502(h)

This delegates to §164.522(b) (a stub).

9.9 §164.502(i) — Notice Consistency

```
permitted_by_164_502_i(ARGS,
    $Step1("164.502(i): consistent with notice
            per 164.520", E)) :-
    permitted_by_164_520(ARGS, E).
```

Listing 52: §164.502(i)

Delegates to §164.520 (a stub).

9.10 §164.502(j) — Whistleblower and Crime Victim Protections

9.10.1 §164.502(j)(1) — Whistleblower

HIPAA Regulation Text

"A covered entity is not considered to have violated the requirements of this subpart if a member of its workforce or a business associate discloses protected health information, provided that (i) The workforce member or business associate believes in good faith that the covered entity has engaged in conduct that is unlawful or otherwise violates professional or clinical standards... and (ii) The disclosure is to [an oversight agency, public health authority, or attorney]."

```
permitted_by_164_502_j_1(ARGS,
    $Leaf("164.502(j)(1): whistleblower -- good faith belief
           of unlawful conduct")) :-
    // p1 is employee or BA of some covered entity CE
    (is_employee_of(p1, CE) ; is_business_associate_of(p1, CE)),
    is_covered_entity(CE),
    // Good faith belief of unlawful/unethical conduct
    believes_unlawful_conduct(p1, CE),
    // Disclosure is to oversight agency, public health authority,
    // or attorney
    (is_oversight_agency(p2) ;
     is_public_health_authority(p2) ;
     activerole(p2, "healthcare-accreditation-org") ;
     activerole(p2, "attorney")),
    is_phi(t),
    msg_contains(m, q, t),
    known_purpose(u).
```

Listing 53: §164.502(j)(1) — whistleblower

Design Decision

Note the existential variable `CE`: the rule says "there exists some covered entity `CE` such that `p1` is an employee or BA of `CE` and `p1` believes `CE` engaged in unlawful conduct." In Soufflé, existentials are expressed naturally by having a variable appear in the body but not the head.

9.10.2 §164.502(j)(2) — Crime Victim

HIPAA Regulation Text

"A covered entity is not considered to have violated the requirements of this subpart if a member of its workforce who is the victim of a criminal act discloses protected health information to a law enforcement official, provided that (i) The PHI disclosed is about the suspected perpetrator; and (ii) The PHI is limited to the information listed in §164.512(f)(2)(i)."

```
permitted_by_164_502_j_2(ARGS,
    $Leaf("164.502(j)(2): crime victim disclosure
           to law enforcement")) :-
    is_employee_of(p1, CE),
```

```
    is_covered_entity(CE),
    believes_victim_of_crime(p1, CE),
    is_law_enforcement(p2),
    is_about_suspected_perpetrator(m, q),
    is_phi(t), msg_contains(m, q, t),
    known_purpose(u).
```

Listing 54: §164.502(j)(2) — crime victim

10 Handling Negation

10.1 Stratified Negation in Soufflé

Soufflé enforces **stratified negation**: the program's dependency graph must have no cycle that passes through a negative edge. This is checked at compile time — unstratifiable programs are rejected.

The key intuition is: *negation-as-failure only makes sense when the thing you are failing on is done being derived.* If relation A negates relation B, then B must be in a strictly lower stratum (computed to a fixed point before A begins).

10.2 Stratification Design in the HIPAA Checker

The compliance checker uses three effective strata:

Stratum	Relations
0	Base facts: `activerole`, `msg_contains`, `disclosure_attempted`, oracle predicates, hierarchy base facts
1	Positive rules: hierarchy closures (`role_subtype`, `attr_subtype`, `purpose_subtype`), helper predicates (`has_role`, `is_covered_entity`, `is_phi`), and stub declarations
2	Negation rules: `permitted_by_164_506_a` (negates authorization), `permitted_by_164_502_e` (negates exclusions), `is_personal_representative` (negates abuse), `is_disclosure_denied` (negates allowed)

Table 3: Stratification design.

10.3 Examples of Negation in the Codebase

1. **§164.506(a)**: `!require_authorization_by_164_508(ARGS)` — "except when authorization is required." Since §164.508 is a stub, this always succeeds.
2. **§164.502(e)(1)(i)**: `!excluded_164_502_e_1_ii(ARGS)` — the business associate rule does not apply when one of the exceptions holds (e.g., disclosure to provider for treatment).
3. **Personal representative**: `!abuse_exception(P, Q)` and `!minor_acts_as_individual(P, Q)` — representative status is blocked if abuse or minor exception applies.

4. **Denial tracking**: `!is_disclosure_allowed(ARGS, _)` — a disclosure is denied if it was attempted but no permission pathway matched. The underscore `_` is a wildcard for the explanation argument.

10.4 Safety Rule for Negation

Every variable in a negated literal must be bound by a positive literal in the same rule body. This is the "safety" requirement:

```
// UNSAFE -- X is not bound by a positive literal
bad(X) :- !edge(X, _).

// SAFE -- X is bound by node(X)
good(X) :- node(X), !edge(X, _).
```

In our codebase, we ensure safety by always including positive grounding literals before negated ones (e.g., `is_covered_entity(p1)` grounds `p1` before `!excluded_164_502_e_1_ii(ARGS)` is evaluated).

11 Handling Temporal Logic

11.1 Why Temporal Logic Matters

The source PDF formalization (HIPAA_FORMALIZATION.pdf) uses first-order temporal logic with operators like:

- $\Box$ (always/globally): "at all future time points"
- $\Diamond$ (eventually/sometimes): "at some past or future time point"
- G (globally, in LTL): used in the top-level formula $G\forall p_1, p_2 : \text{prin}.\ \forall m : \text{msg}.\ \text{send}(p_1, p_2, m) \supset (\bigvee_i \varphi_i^+ \wedge \bigwedge_i \varphi_i^-)$

For example, §164.506(c)(4) uses $\Diamond\texttt{inrelationship}(p_1, r_1, q)$ to express "entity has or had a relationship with the subject."

11.2 Why Kamp-Style Translation Was Deferred

Kamp's theorem states that any formula in first-order temporal logic over linear time can be translated to a pure first-order formula using explicit time variables. For example:

$$\Diamond\texttt{inrelationship}(p_1, r_1, q) \quad \Longrightarrow \quad \exists t'.\ t' \leq t \wedge \texttt{inrelationship\_at}(p_1, r_1, q, t')$$

However, for §164.502 and §164.506:

- Most rules are **time-free**: they state static conditions ("if the entity is covered and the info is PHI and the purpose is treatment, then the disclosure is permitted").
- The few temporal references (like "has or had a relationship") can be conservatively approximated by atemporal predicates.
- Full temporal infrastructure would require a `TimePoint` domain, timestamped facts, and explicit time comparisons in every rule — adding complexity without benefit for these two sections.

11.3 Oracle Predicates as Temporal Substitutes

For the temporal operators in §164.502/506, we use oracle predicates:

- `inrelationship(p1, r, q)`: asserted as a fact if the relationship exists (or ever existed).
- `has_authority_to_act(p, q)`: asserted if authority exists at the current time.

11.4 Future Temporal Infrastructure

When formalizing sections with heavier temporal content (e.g., §164.508 authorization validity periods, §164.530 retention requirements), the following infrastructure would be needed:

```
.type TimePoint <: number  // already declared in hipaa_types.dl

// Timestamped facts
.decl inrelationship_at(entity: Principal, rel: Rel,
    subject: Principal, t: TimePoint)

// Current time (single-fact relation)
.decl current_time(t: TimePoint)

// "Has or had" becomes existential over time
.decl has_or_had_relationship(entity: Principal, rel: Rel,
    subject: Principal)
has_or_had_relationship(E, R, S) :-
    inrelationship_at(E, R, S, T),
    current_time(Now),
    T <= Now.
```

Listing 55: Sketch of temporal infrastructure

12 Stub Pattern for Unformalized Sections

12.1 The Problem

Sections §164.502 and §164.506 reference many other sections (§164.508, §164.510, §164.512, §164.514, §164.520, §164.522, §164.524, §164.528, §164.530, §164.504, §160.C). If these are left undeclared, Soufflé will produce a compile error when a rule references an undeclared relation.

12.2 The Solution: Empty Declarations

We declare each referenced relation but give it *no rules*:

```
// 164.508: Authorization required
.decl require_authorization_by_164_508(DECL_ARGS)
.decl obtained_authorization_164_508(p1:Principal, p2:Principal,
    q:Principal, t:Attribute, u:Purpose)

// 164.510: Opportunity to agree/object
.decl permitted_by_164_510(DECL_ARGS, e: Expl)

// 164.512: No authorization or opportunity required
.decl permitted_by_164_512(DECL_ARGS, e: Expl)
.decl required_by_164_512_a(DECL_ARGS)

```

```
// 164.514: De-identification standards
.decl permitted_by_164_514_d(DECL_ARGS)

// 164.520: Notice of privacy practices
.decl permitted_by_164_520(DECL_ARGS, e: Expl)

// 164.522: Rights to request restriction
.decl permitted_by_164_522_a_1(DECL_ARGS)
.decl permitted_by_164_522_a(DECL_ARGS)
.decl permitted_by_164_522_b(DECL_ARGS)

// 164.524: Access of individuals to PHI
.decl required_by_164_524(DECL_ARGS)

// 164.528: Accounting of disclosures
.decl required_by_164_528(DECL_ARGS)

// 160.C: Subpart C of Part 160
.decl permitted_by_160_C(DECL_ARGS)
```

Listing 56: Examples from hipaa_stubs.dl

12.3 Conservative Default

Because these relations have no rules and no facts, they are **empty**. Under the closed-world assumption, this means:

- **Positive references fail**: any rule that requires a stub relation in its body will never fire. For example, `permitted_by_164_502_a_1_v` requires `permitted_by_164_510(ARGS, E)`, which is always empty. So no disclosures are permitted via §164.510 until it is formalized.
- **Negated references succeed**: any rule that negates a stub relation will always succeed on the negation. For example, `!require_authorization_by_164_508(ARGS)` always holds because the stub is empty.

This is a **conservative default**: we err on the side of not permitting disclosures through unformalized pathways. The one exception is negated stubs, which are permissive (the "except" clause does not block). This matches the regulatory intent: the default is prohibition, and specific sections provide exceptions.

13 Grounding Issues

13.1 The Grounding Requirement

Soufflé requires that every variable in a rule body must appear in at least one positive (non-negated) body literal. This is called the "safety" or "grounding" requirement. It ensures that the engine can enumerate all possible values of the variable.

13.2 Problem: Unrestricted Purpose Variable

Several HIPAA rules do not restrict the purpose of disclosure. For example, §164.502(a)(1)(i) says disclosures "to the individual" are permitted for *any* purpose. But the rule head includes the purpose variable `u`, so it must be grounded:

```
// This would be UNSAFE -- u is not grounded:
// permitted_by_164_502_a_1_i(ARGS, ...) :-
//     is_covered_entity(p1), is_phi(t), msg_contains(m, q, t),
//     (p2 = q ; is_personal_representative(p2, q)).
//     // ERROR: u is not bound by any positive literal!
```

13.3 Solution: known_purpose(u)

We define a `known_purpose` relation that contains all purpose values:

```
.decl known_purpose(u: Purpose)
known_purpose(U) :- purpose_isa(U, _).
known_purpose(U) :- purpose_isa(_, U).
known_purpose("treatment").
known_purpose("marketing").
known_purpose("research").
// ... all top-level purposes
```

Then use it to ground `u`:

```
permitted_by_164_502_a_1_i(ARGS, ...) :-
    is_covered_entity(p1), is_phi(t), msg_contains(m, q, t),
    known_purpose(u),  // grounds u
    (p2 = q ; is_personal_representative(p2, q)).
```

13.4 Problem: Unrestricted Disclosure in (d)(2)

§164.502(d)(2) says de-identified information may be freely disclosed. But "freely" means the sender, receiver, purpose, and message are all unrestricted. We use `disclosure_attempted(ARGS)` to ground all variables:

```
permitted_by_164_502_d(ARGS, ...) :-
    is_covered_entity(p1),
    is_dii(t),
    msg_contains(m, q, t),
    disclosure_attempted(ARGS).  // grounds p2 and u
```

This means de-identified info is "freely disclosed" only for attempted disclosures — we do not generate phantom disclosures.

14 Prolog to Soufflé Translation

The PDF formalization uses Prolog-like syntax. The following table maps Prolog constructs to their Soufflé equivalents:

Prolog (PDF)	Soufflé	Notes
`fail`	Omit the rule	A rule that always fails is simply not written.
`writeln('...')`	`$Leaf("...")`	Debug output becomes ADT explanation reason string.
`\+` (negation)	`!`	Prolog negation-as-failure becomes Soufflé stratified negation.
`;` (disjunction)	`;` or multiple rules	Soufflé supports `;` in rule bodies, or use separate rules for the same head.
`debug(...)`	Comment	PDF debug calls become comments in the Soufflé code.
Bundled `A`	`(p1, p2, q, m, t, u)`	The single parameter is unpacked into 6 variables.
`is_from_coveredEntity(A)`	`is_covered_entity(p1)`	Macro expanded to check `p1`'s role.
`is_phi(A)`	`is_phi(t)`	Macro expanded to check attribute type.
`is_for_either-Purpose(A)`	`is_for_tpo(u)`	Macro renamed and expanded.
`is_to_health-CareProvider(A)`	`is_health_care_provider(p2)`	Macro expanded to check `p2`'s role.
`is_to_concerned-Individual(A)`	(p2 = q ; is_personal_representative(p2, q))	"Concerned individual" unpacked.
`activerole(p, role)`	`activerole(P, "role")`	Direct or hierarchy-aware lookup via `has_role`.
`contains(m, (q, t))`	`msg_contains(m, q, t)`	Renamed (reserved word in Soufflé).

Table 4: Prolog to Soufflé translation reference.

15 Test Scenarios

The fact database (`hipaa_facts.dl`) defines 10 test scenarios. Each scenario is a `disclosure_attempted` fact with an expected outcome.

15.1 Principals and Roles

Principal	Role	Category
`mercy_hospital`	hospital	Covered entity
`blue_cross`	health-insurance-issuer	Covered entity
`dr_smith`	doctor	Provider (member of mercy_hospital)
`dr_jones`	psychiatrist	Provider (member of mercy_hospital)
`nurse_adams`	nurse	Provider (employee of mercy_hospital)
`lab_corp`	lab-technician	Provider
`patient_alice`	patient	Adult
`patient_bob`	patient	Adult
`patient_charlie`	patient	Unemancipated minor
`secretary_hhs`	secretary	HHS Secretary
`state_health_dept`	oversight-agency	Government
`fbi_agent_davis`	law-enforcement-official	Government
`billing_co`	billing-company	Business associate (of mercy_hospital)
`random_person`	individual	No covered-entity role
`attorney_wilson`	attorney	Attorney
`parent_frank`	parent	Guardian of patient_charlie

Table 5: Test principals.

15.2 The 10 Scenarios

#	Scenario	Expected	HIPAA Section
1	Hospital sends blood test results to doctor for treatment	ALLOW	§506(c)(2)
2	Hospital sends diagnosis directly to patient	ALLOW	§502(a)(1)(i)
3	Hospital sends billing record to insurer for payment	ALLOW	§506(c)(3)
4	Hospital sends diagnosis to random person for marketing	DENY	No pathway matches
5	Whistleblower nurse discloses to oversight agency	ALLOW	§502(j)(1)
6	Hospital sends PHI to Secretary for investigation	ALLOW	§502(a)(2)(ii)
7	Hospital discloses billing record to business associate	ALLOW	§502(e)(1)(i)
8	De-identified info (statistical data) to random person	ALLOW	§502(d)(2)
9	Parent requests PHI about minor child	ALLOW	§502(a)(1)(i) + personal rep
10	Hospital to insurer for healthcare operations (quality assessment)	ALLOW	§506(c)(4)

Table 6: Test scenarios with expected outcomes.

15.3 Scenario Analysis

15.3.1 Scenario 1: Hospital to Doctor for Treatment

```
disclosure_attempted("mercy_hospital", "dr_smith", "patient_alice",
    "msg_001", "blood-test-results", "treatment").
msg_contains("msg_001", "patient_alice", "blood-test-results").
```

Listing 57: Scenario 1 facts

Expected derivation path:

1. `is_covered_entity("mercy_hospital")` via `has_role` (hospital < covered-entity).
2. `is_phi("blood-test-results")` via `attr_subtype` (blood-test-results < phi).
3. `is_for_treatment("treatment")` directly.
4. `is_health_care_provider("dr_smith")` via `has_role` (doctor < provider).
5. `permitted_by_164_506_c_2` fires.
6. Propagates up through `permitted_by_164_506_c` → `permitted_by_164_506_a` → `permitted_by_164_506` → `permitted_by_164_502_a_1_ii` → `permitted_by_164_502_a_1` → `permitted_by_164_502_a` → `permitted_by_164_502` → `is_disclosure_allowed`.

15.3.2 Scenario 4: Hospital to Random Person for Marketing (DENY)

```
disclosure_attempted("mercy_hospital", "random_person", "patient_alice",
    "msg_004", "diagnosis", "marketing").
```

Listing 58: Scenario 4 facts

Why this is denied:

- §502(a)(1)(i): `random_person` ≠ `patient_alice` and is not a personal representative.
- §502(a)(1)(ii): purpose is "marketing," not TPO.
- §502(a)(1)(iii): no `is_incident_to_use` fact.
- §502(a)(1)(iv): no authorization obtained.
- §502(a)(1)(v): §164.510 is a stub.
- §502(a)(1)(vi): §164.512, 514(e/f/g) are stubs.
- §502(c/d/e/h/i/j): none of the special pathways match.

No permission pathway fires, so `is_disclosure_denied` is derived via the closed-world assumption.

15.3.3 Scenario 9: Parent Requests PHI About Minor Child

```
disclosure_attempted("mercy_hospital", "parent_frank", "patient_charlie",
    "msg_009", "diagnosis", "treatment").
is_guardian("parent_frank", "patient_charlie").
has_authority_to_act("parent_frank", "patient_charlie").
belongstorole("patient_charlie", "unemancipated-minor").
```

Listing 59: Scenario 9 facts

Expected derivation:

1. `is_personal_representative("parent_frank", "patient_charlie")` via §502(g)(3): parent_frank is a guardian, patient_charlie is an unemancipated minor, authority exists, no abuse or minor exception.
2. `permitted_by_164_502_a_1_i` fires because `is_personal_representative(p2, q)` holds.
3. The disclosure is allowed via §502(a)(1)(i).

16 How to Run Everything

16.1 Step 1: Install Soufflé

```
brew install souffle
souffle --version  # verify installation
```

16.2 Step 2: Navigate to the Project Directory

```
cd /path/to/ComplianceGPT/datalog_engine
```

16.3 Step 3: Run the Compliance Checker

Option A — use the wrapper script:

```
chmod +x run_hipaa.sh
./run_hipaa.sh
```

Option B — run Soufflé directly:

```
mkdir -p output_hipaa
souffle -D output_hipaa hipaa_main.dl
```

16.4 Step 4: Interpret the Output

The output directory `output_hipaa/` contains CSV files:

`is_disclosure_allowed.csv`: Each row is a 7-tuple: sender, receiver, subject, message, attribute, purpose, and explanation tree. If a disclosure attempt appears here, it is allowed.

`is_disclosure_denied.csv`: Each row is a 6-tuple (no explanation). If a disclosure attempt appears here, it was denied.

`disclosure_attempted.csv`: All attempted disclosures (for verification).

`permitted_by_164_502.csv`: Intermediate results showing which §502 pathway matched.

`permitted_by_164_506.csv`: Intermediate results showing which §506 pathway matched.

`minimum_necessary_satisfied.csv`: Which disclosures satisfy minimum necessary.

`is_personal_representative.csv`: Derived personal representative relationships.

`excluded_164_502_b_2.csv`: Minimum necessary exceptions.

16.5 Step 5: Use Explain Mode for Debugging

```
souffle -t explain hipaa_main.dl
```

At the prompt:

```
> explain is_disclosure_allowed("mercy_hospital", "dr_smith",
    "patient_alice", "msg_001", "blood-test-results", "treatment", _)
```

This displays the full proof tree showing which rules and facts were used.

For denied disclosures:

```
> explainnegation is_disclosure_allowed("mercy_hospital",
    "random_person", "patient_alice", "msg_004",
    "diagnosis", "marketing", _)
```

This interactively walks through the derivation attempt and shows where it fails.

16.6 Step 6: Add New Test Scenarios

To add a new scenario, edit `hipaa_facts.dl` and add:

1. Any new principal declarations (`activerole`).
2. Message contents (`msg_contains`).
3. Oracle predicates if needed.
4. A `disclosure_attempted` fact.

Then re-run the checker.

17 Lessons Learned and Debugging Tips

17.1 Reserved Words

Soufflé has reserved words that cannot be used as relation names. We discovered that `contains` is reserved — the PDF's `contains(m, (q, t))` had to be renamed to `msg_contains(m, q, t)`.

Other reserved words to watch for: `input`, `output`, `type`, `decl`, `number`, `symbol`, `float`, `unsigned`, `nil`, `count`, `sum`, `min`, `max`, `mean`, `cat`, `ord`, `strlen`, `substr`, `match`, `true`, `false`.

17.2 Grounding Errors

The most common compile error during development was ungrounded variables. The fix is always the same: find a positive relation that contains the variable and add it to the rule body. We used:

- `known_purpose(u)` for unrestricted purpose.
- `disclosure_attempted(ARGS)` for fully unrestricted disclosures.
- `msg_contains(m, q, t)` to ground message, subject, and attribute simultaneously.

17.3 Stratification Errors

If you add a rule that creates a negation cycle, Soufflé will report a compile error like:

```
Error: Unable to stratify program
  Negative recursion involving relation X
```

To debug:

1. Use `souffle -show=precedence-graph` to visualize the dependency graph.
2. Find the cycle involving a negative edge.
3. Refactor so the negated relation is in a lower stratum.

17.4 ADT Constructor Names Must Be Globally Unique

In Soufflé, ADT branch names must be unique across *all* ADTs in the program. If you define another ADT with a branch named `Leaf`, it will conflict with the `Expl` ADT's `Leaf`. Always use distinctive names.

17.5 Disjunction Gotchas

Soufflé's `;` (disjunction) in rule bodies requires careful use:

- Parentheses are mandatory around disjunctions: `(A ; B)`.
- Variables bound in one branch of a disjunction are *not* bound in the other. Each branch must independently ground all its variables.
- When in doubt, split into multiple rules instead of using `;`.

17.6 Empty Stubs and Unintended Blocking

Remember that stub relations are empty, so:

- A rule requiring a stub in its positive body *will never fire*.
- A rule negating a stub *will always succeed on the negation*.

This can cause subtle bugs. For example, when we first wrote §164.502(a)(2)(ii), including `permitted_by_160_C(ARGS)` in the body blocked the rule entirely (because §160.C is stubbed). The fix was to replace it with an oracle predicate `secretary_investigation_authorized(ARGS)`.

17.7 The Closed-World Assumption and Denial

Soufflé uses the **closed-world assumption**: any tuple not derivable is considered false. This is exactly what we need for HIPAA compliance: "not explicitly allowed" means "denied." The `is_disclosure_denied` relation makes this explicit:

```
1 is_disclosure_denied(ARGS) :-
2     disclosure_attempted(ARGS),
3     !is_disclosure_allowed(ARGS, _).
```

A disclosure is denied if and only if it was attempted and no permission pathway matched. There is no need to enumerate denial reasons — the *absence* of a permission is the denial.

18 Formalization of §164.508 — Authorization Required

18.1 What is Authorization?

In HIPAA, **authorization** is a detailed, written permission signed by the individual (patient) that allows a covered entity to use or disclose their PHI for a specific purpose. It is more formal than consent: an authorization must contain specific elements (a description of the information, who may use it, the purpose, an expiration date, the individual's signature, and more). Think of it as a signed contract granting permission for a specific disclosure.

Most disclosures under HIPAA do *not* require authorization — TPO disclosures (§164.506), public health disclosures (§164.512(b)), and many others are permitted without one. Section 164.508 identifies the specific categories where authorization *is* required, and blocks disclosures in those categories unless the authorization has been obtained.

18.2 The Negative Norm Concept

Section 164.508 is modeled as a **negative norm** — it does not independently permit disclosures. Instead, it **blocks** disclosures that would otherwise be permitted by §164.506 (TPO). Specifically:

- **Psychotherapy notes** (§508(a)(2)): Even though a disclosure might qualify as TPO under §506, if the information is psychotherapy notes, an authorization is required.
- **Marketing** (§508(a)(3)): If PHI is being used for marketing purposes, an authorization is required.

In the Datalog formalization, this blocking is implemented through the predicate `require_authorization_by_164_508`. When this predicate holds for a given disclosure, it means "this disclosure needs an authorization that has not been obtained." The §506(a) rule checks for the *absence* of this predicate using negation:

```
// In hipaa_164_506.dl:
permitted_by_164_506_a(ARGS, ...) :-
    !require_authorization_by_164_508(ARGS),  // 508 does NOT block
    is_covered_entity(p1),
    is_for_tpo(u),
    permitted_by_164_506_c(ARGS, E).
```

Listing 60: How §508 blocks §506(a) via negation

The `!` (exclamation mark) means "NOT." So this rule says: "TPO disclosure is permitted if §508 does *not* require authorization for it, AND it meets the §506(c) implementation specification."

18.3 Structure of the Negative Norm

The `require_authorization_by_164_508` predicate fires when authorization is required but has *not* been obtained, and no exception applies:

```
// Psychotherapy notes require authorization (unless exception)
require_authorization_by_164_508(ARGS) :-
    is_covered_entity(p1),
    is_psychotherapy_notes(t),
    msg_contains(m, q, t),
    known_purpose(u),
    disclosure_attempted(ARGS),
    !exception_508a2(ARGS),
    !obtained_authorization_164_508(p1, p2, q, t, u).

// Marketing requires authorization (unless exception)
require_authorization_by_164_508(ARGS) :-
    is_covered_entity(p1),
    is_phi(t),
    msg_contains(m, q, t),
    is_for_marketing(u),
    disclosure_attempted(ARGS),
    !exception_508a3(ARGS),
    !obtained_authorization_164_508(p1, p2, q, t, u).
```

Listing 61: The negative norm — require_authorization (from hipaa_164_508.dl)

Read this rule carefully: the predicate fires ("authorization is required") when:

1. The sender is a covered entity.
2. The information is psychotherapy notes (or the purpose is marketing).
3. No exception from §508(a)(2) or (a)(3) applies.
4. The individual has *not* provided authorization.

If the individual *has* provided authorization, `obtained_authorization_164_508` holds, its negation fails, and the `require_authorization` predicate does *not* fire — meaning §506(a) is not blocked.

18.4 Exception Predicates

Several exceptions can bypass the authorization requirement:

§508(a)(2)(i)(B) — CE training: A covered entity may use psychotherapy notes for its own counseling training programs without authorization.

§508(a)(2)(i)(C) — Legal defense: A covered entity may use psychotherapy notes to defend itself in a legal proceeding brought by the individual.

§508(a)(2)(ii) — Cross-references: Psychotherapy notes may be disclosed without authorization if permitted by §512(a) (required by law), §512(d) (health oversight), §512(g)(1) (coroners), or §512(j)(1)(i) (serious threat).

§508(a)(3)(i)(A) — Face-to-face: Marketing via face-to-face communication does not require authorization.

§508(a)(3)(i)(B) — Nominal gifts: Marketing involving a promotional gift of nominal value does not require authorization.

18.5 Example: Psychotherapy Notes for Payment

Consider this scenario: a hospital wants to disclose a patient's psychotherapy notes to an insurer for payment. Under §506(c)(3), a covered entity may disclose PHI to another covered entity for the receiver's payment activities. Payment is a TPO purpose, so §506 would normally permit this.

However, psychotherapy notes trigger §508(a)(2). The authorization requirement fires because:

- The information is psychotherapy notes.
- None of the exceptions (training, legal defense, cross-references) apply to payment.
- No authorization was obtained.

So `require_authorization_by_164_508` holds, its negation in §506(a) fails, and the disclosure is blocked. This is the correct result — HIPAA gives psychotherapy notes extra protection, even for payment.

Design Decision

There is a design subtlety here. The §506 rules are structured so that §506(a) is the "standard" pathway that checks the §508 negative norm, while §506(c)(3) provides the implementation specification. Because §506(a) is the gatekeeper that checks

`!require_authorization_by_164_508`, and §506(c)(3) is only reachable through §506(a), the authorization check applies to all TPO disclosures — including payment for psychotherapy notes.

19 Formalization of §164.510 — Opportunity to Agree or Object

19.1 Overview

Section 164.510 covers two situations where a covered entity may disclose PHI if the individual is given an opportunity to agree or object (or, in emergencies, if certain conditions are met):

§510(a) — Facility directories: Hospitals and similar facilities may maintain a directory of patients (name, location in facility, general condition, religious affiliation) and share it under specific rules.

§510(b) — Care involvement and notification: A covered entity may disclose PHI to family members, close friends, or others involved in the individual's care, or for notification purposes (e.g., notifying a family member about a patient's location).

The top-level rule simply combines these two sub-sections:

```
permitted_by_164_510(ARGS, ...) :-
    is_phi(t), msg_contains(m, q, t),
    permitted_by_164_510_a(ARGS, E).   // facility directory

permitted_by_164_510(ARGS, ...) :-
    is_phi(t), msg_contains(m, q, t),
    permitted_by_164_510_b(ARGS, E).   // care involvement
```

Listing 62: Top-level §510 (from hipaa_164_510.dl)

19.2 Facility Directories (§510(a))

A hospital facility directory typically contains: the patient's name, location in the facility (e.g., "Room 302"), condition in general terms (e.g., "stable"), and religious affiliation. The rules for who can access this directory depend on who is asking:

Clergy: Members of the clergy may receive *all* directory information, including religious affiliation. This allows chaplains and religious leaders to visit patients of their faith.

Others asking by name: Anyone who asks for a patient by name may receive directory information *except* religious affiliation. The idea is that if someone already knows the patient's name, they can find out the room number and general condition, but religious affiliation is more sensitive.

```
// Clergy: all directory info (including religious affiliation)
permitted_by_164_510_a(ARGS, ...) :-
    is_covered_entity(p1),
    is_directory_info(t),
    msg_contains(m, q, t),
    is_for_directory(u),
    is_clergy(p2),
    has_not_objected_to_directory(p1, p2, q, t, u).
```

```
9
10 // Others asking by name: directory info MINUS religious affiliation
11 permitted_by_164_510_a(ARGS, ...) :-
12     is_covered_entity(p1),
13     is_directory_info(t),
14     t != "religious-affiliation",       // <-- key difference
15     msg_contains(m, q, t),
16     is_for_directory(u),
17     is_directory_request_by_name(p2, p1, q, t, u),
18     has_not_objected_to_directory(p1, p2, q, t, u).
```

Listing 63: Clergy vs. by-name directory lookup (from hipaa_164_510.dl)

Notice the `t != "religious-affiliation"` guard in the second rule. This is a simple but effective way to exclude one attribute type from an otherwise-broad permission.

Both rules also require `has_not_objected_to_directory` — an oracle predicate meaning the patient was given an opportunity to object and did not. If the patient objects, their information is removed from the directory.

19.3 Incapacity and Emergency Exceptions (§510(a)(3))

What if the patient is unconscious or in emergency treatment and cannot be asked whether they object? The formalization handles this with a separate rule:

```
1 permitted_by_164_510_a(ARGS, ...) :-
2     is_covered_entity(p1),
3     is_directory_info(t),
4     msg_contains(m, q, t),
5     is_for_directory(u),
6     (belongs_to(q, "incapacitated") ; belongs_to(q, "emergency-treatment")),
7     consistent_with_prior_preference(p1, p2, q, t, u),
8     believes_in_best_interest_directory(p1, p2, q, t, u).
```

Listing 64: Emergency exception for directory (§510(a)(3))

This rule permits directory disclosure when the patient is incapacitated or in emergency treatment, provided the disclosure is consistent with any previously expressed preference and the covered entity believes it is in the patient's best interest. Both of these conditions are oracle predicates — they represent professional judgment calls that cannot be derived from the data alone.

19.4 Care Involvement (§510(b))

Section 510(b) allows disclosure to people involved in the patient's care. The key question is: who counts as a valid recipient?

```
1 // Care involvement recipient check
2 is_care_involvement_recipient(P2, Q) :- is_family_member(P2, Q).
3 is_care_involvement_recipient(P2, Q) :- is_close_personal_friend(P2, Q).
4 is_care_involvement_recipient(P2, Q) :- is_identified_by_individual(P2, Q).
5 is_care_involvement_recipient(P2, Q) :- is_personal_representative(P2, Q).
```

Listing 65: Who can receive care involvement disclosures

A disclosure to a care involvement recipient is permitted if:

1. The information is **directly relevant** to that person's involvement in the patient's care (`relevant_to_involvement`).
2. The patient has agreed or not objected (§510(b)(2)), OR the patient is not present/incapacitated and the CE exercises professional judgment (§510(b)(3)).

Example. A doctor tells a patient's spouse about the patient's medication schedule so the spouse can help manage the patient's care at home. The spouse is a family member (`is_family_member`), the medication information is relevant to care involvement (`relevant_to_involvement`), and the patient was informed and did not object. This satisfies §510(b)(1)(i) and §510(b)(2).

Notification (§510(b)(1)(ii)). A special sub-rule allows disclosure of *location, condition, or death* information for notification purposes. This is how hospitals can call a family member to say "your relative is in the ER" without violating HIPAA.

19.5 Disaster Relief (§510(b)(4))

A covered entity may disclose PHI to entities authorized for disaster relief (like the Red Cross) to coordinate family notification during disasters, subject to the same agree/object framework or professional judgment.

20 Formalization of §164.512 — Uses and Disclosures Without Authorization or Agreement

20.1 Overview

Section 164.512 is the largest section of the Privacy Rule. It enumerates twelve categories of disclosures that require *neither* patient authorization (§508) *nor* an opportunity to agree or object (§510). These are generally public-interest or legally-mandated disclosures.

The top-level structure is a simple disjunction — if *any* sub-section permits the disclosure, then §512 permits it:

```
permitted_by_164_512(ARGS, E) :- permitted_by_164_512_a(ARGS, E).
permitted_by_164_512(ARGS, E) :- permitted_by_164_512_b(ARGS, E).
permitted_by_164_512(ARGS, E) :- permitted_by_164_512_c(ARGS, E).
// ... through (l)
permitted_by_164_512(ARGS, E) :- permitted_by_164_512_l(ARGS, E).
```

Listing 66: Top-level §512 disjunction (from hipaa_164_512.dl)

20.2 Sub-section (a): Required by Law

If a disclosure is required by another law (federal, state, or local), HIPAA permits it. This is the simplest sub-section — a single oracle predicate determines whether a legal requirement exists:

```
permitted_by_164_512_a(ARGS, $Leaf("164.512(a): required by law")) :-
    is_covered_entity(p1),
    is_phi(t),
    msg_contains(m, q, t),
    is_required_by_law(p1, p2, q, t, u).  // oracle predicate
```

Listing 67: §512(a): Required by law

The predicate `is_required_by_law` is an oracle: the system cannot derive from its internal data whether some external law mandates the disclosure. This must be asserted as a fact in the test scenario.

20.3 Sub-section (b): Public Health Activities

Permits disclosures to public health authorities and related entities. Five sub-rules cover:

(b)(1)(i) To a public health authority for disease prevention/control.

(b)(1)(ii) Child abuse reports to authorized authorities.

(b)(1)(iii) FDA-regulated product quality, safety, and effectiveness activities.

(b)(1)(iv) Notification to a person at risk of contracting or spreading a disease.

(b)(1)(v) Workplace medical surveillance to employers (with prior notice to the individual).

Each sub-rule follows the same pattern: check that the sender is a covered entity, the information is PHI, the recipient has the appropriate role, and the purpose matches. Most conditions involve oracle predicates (e.g., `is_authorized_by_law_for_purpose`, `has_given_notice_of_workplace_disclosure`).

20.4 Sub-section (c): Abuse, Neglect, or Domestic Violence

Permits reporting abuse victims to government authorities. The disclosure is allowed when: (1) it is required by law, (2) the individual agrees, or (3) the CE is authorized by statute and believes disclosure is necessary to prevent harm. These three alternatives are combined with disjunction:

```
permitted_by_164_512_c(ARGS, ...) :-
    is_covered_entity(p1),
    has_role(p2, "government-authority"),
    believes_victim_of_abuse(p1, q),
    u = "reports-of-abuse",
    (is_required_by_law(p1, p2, q, t, u) ;
     individual_has_agreed_to_disclosure(p1, p2, q, t, u) ;
     (authorized_by_statute_regulation(p1, p2, q, t, u),
      believes_disclosure_necessary_to_prevent_harm(p1, p2, q, t, u))).
```

Listing 68: §512(c): Three alternative conditions combined with ; (OR)

20.5 Sub-section (d): Health Oversight Activities

Permits disclosures to health oversight agencies (e.g., HHS Office for Civil Rights, state licensing boards) for audits, investigations, and inspections authorized by law.

20.6 Sub-section (e): Judicial and Administrative Proceedings

Permits disclosures in response to: (i) court orders, (ii) subpoenas with satisfactory assurances of notice to the individual, or (vi) reasonable efforts to notify. Each has its own oracle predicate (e.g., `has_court_order`, `has_lawful_process_with_assurance`).

20.7 Sub-section (f): Law Enforcement

The most complex sub-section, with six sub-rules:

(f)(1) Required by law or court order/warrant/subpoena to law enforcement.

(f)(2) Limited identifying information (name, address, SSN, blood type, injury type, etc.) to identify or locate a suspect.

(f)(3) Crime victim disclosure — with the individual's agreement, or in emergencies when agreement cannot be obtained.

(f)(4) Suspicious death notification.

(f)(5) Evidence of crime on the CE's premises.

(f)(6) Medical emergency — alerting law enforcement about a crime, but *not* if the emergency is believed to result from abuse (a negative norm within a positive norm).

The (f)(6) rule illustrates a negative norm nested inside a positive norm:

```
permitted_by_164_512_f_6(ARGS, ...) :-
    is_health_care_provider(p1),
    is_law_enforcement(p2),
    providing_emergency_healthcare(p1, q),
    appears_necessary_to_alert_crime(p1, p2, q, t, u),
    !believes_emergency_result_of_abuse(p1, q).  // blocks if abuse
```

Listing 69: §512(f)(6): Nested negative norm

20.8 Sub-section (g): Decedents

Permits disclosures about deceased individuals to: (g)(1) coroners and medical examiners for identification or cause-of-death determination, and (g)(2) funeral directors for carrying out their duties. For funeral directors, the rule also covers individuals whose death is "reasonably anticipated."

20.9 Sub-section (h): Organ, Eye, or Tissue Donation

Permits disclosures to organ procurement organizations to facilitate transplantation. This is a straightforward single rule with the recipient role check `is_organ_procurement_entity(p2)`.

20.10 Sub-section (i): Research

Permits research disclosures under three conditions:

(i)(1)(i) An IRB or privacy board has approved a waiver of authorization.

(i)(1)(ii) The researcher represents the PHI is needed only for preparatory research activities (no PHI leaves the CE).

(i)(1)(iii) Research on a decedent's information, with the researcher representing that the PHI is needed for research on the deceased.

Each uses an oracle predicate to represent the researcher's representations or board approval.

20.11 Sub-section (j): Serious Threat to Health or Safety

Permits disclosures to prevent or lessen a serious and imminent threat. Two main pathways:

(j)(1)(i) Disclosure to someone the CE believes can prevent or lessen the threat, consistent with applicable law and professional judgment.

(j)(1)(ii) Disclosure to law enforcement to identify or apprehend an individual who has: (A) admitted to a violent crime (but not information learned through treating propensity for crime), or (B) escaped lawful custody.

20.12 Sub-section (k): Specialized Government Functions

Six sub-rules covering military/veterans, national security, protective services for the President, State Department medical suitability, correctional institutions, and government programs providing public benefits. These are heavily oracle-dependent, as they involve classified or specialized government determinations.

20.13 Sub-section (l): Workers' Compensation

Permits disclosures authorized by workers' compensation laws. A single oracle predicate `authorized_for_workers_comp` determines whether the disclosure is authorized.

20.14 The Oracle Predicate Pattern in §512

A recurring theme throughout §512 is the heavy use of oracle predicates. Nearly every sub-section requires at least one contextual determination that cannot be derived from the data alone:

- `is_required_by_law` — whether another law mandates disclosure.
- `is_authorized_by_law_for_purpose` — whether a recipient is legally authorized.
- `believes_victim_of_abuse` — a subjective professional determination.
- `has_irb_or_privacy_board_waiver` — an external approval process.
- `consistent_with_applicable_law` — a legal compliance determination.

These oracle predicates are the interface between the formal Datalog rules and the real-world context. When testing, you assert them as facts to represent the circumstances of the scenario. When deploying the compliance checker, they would be populated by an external system (e.g., a case management system, a legal database, or a human compliance officer).

21 Formalization of §164.514 — Other Requirements

21.1 Overview

Section 164.514 collects several cross-cutting requirements that apply across the Privacy Rule. Unlike the previous sections, which each describe a specific permission pathway, §514 describes **conditions and constraints** that other permission pathways may reference.

21.2 De-identification Standards (§514(a)–(c))

HIPAA defines "de-identified" health information as information that does not identify an individual and for which there is no reasonable basis to believe it can identify an individual. De-identified information is *not* PHI and therefore not subject to the Privacy Rule.

Section 514(b) specifies two methods for de-identification:

Expert determination (§514(b)(1)): A qualified statistical expert determines that the risk of identification is very small. Modeled as the oracle predicate `expert_determination_deidentified(t)`.

Safe harbor (§514(b)(2)): Eighteen specific types of identifiers are removed (names, geographic data smaller than a state, dates more specific than year, phone numbers, SSNs, etc.). Modeled as the oracle predicate `safe_harbor_deidentified(t)`.

Section 514(c) addresses re-identification codes: a CE may assign a code to de-identified data for re-identification purposes, provided the code is not derived from PHI and the mechanism is not disclosed.

In practice, these are pre-classification constraints — our model classifies attributes as de-identified information (`dii`) at the type level, and these predicates serve as validation oracles.

21.3 Minimum Necessary Implementation (§514(d))

The "minimum necessary" standard requires covered entities to make reasonable efforts to limit PHI disclosures to the minimum necessary to accomplish the purpose. Section 514(d) specifies *how* to implement this:

```
permitted_by_164_514_d(ARGS) :-
    is_covered_entity(p1),
    is_phi(t),
    msg_contains(m, q, t),
    identifies_workforce_needing_phi(p1),      // oracle
    reasonably_limits_phi_access(p1),           // oracle
    implements_policies_for_routine_disclosures(p1),  // oracle
    implements_criteria_for_limiting_phi(p1),   // oracle
    meets_policies_and_criteria(p1, p2, q, t, u).    // oracle
```

Listing 70: Minimum necessary implementation (§514(d))

All five conditions are oracle predicates because they represent organizational policies and practices that the Datalog engine cannot inspect. The predicate `permitted_by_164_514_d` is used as a guard in §502(b) — it represents the CE's compliance with the minimum necessary standard for this specific disclosure.

21.4 Limited Data Sets (§514(e))

A **limited data set** is PHI with 16 types of direct identifiers removed (but more than de-identified data — it can still contain dates, city, state, zip code, and age). A CE may disclose a limited data set for research, public health, or healthcare operations, provided:

1. The data qualifies as a limited data set (`is_limited_data_set(t)`).
2. The purpose is research, public health, or healthcare operations.

3. A **data use agreement** is in place with the recipient (`has_limited_data_use_agreement`).

```
permitted_by_164_514_e(ARGS, ...) :-
    is_covered_entity(p1),
    is_phi(t), msg_contains(m, q, t),
    is_limited_data_set(t),
    is_for_limited_data_set_purpose(u),  // research, public health, or hc-ops
    has_limited_data_use_agreement(p1, p2, q, t, u).
```

Listing 71: Limited data set disclosure (§514(e))

21.5 Fundraising (§514(f))

A covered entity may disclose *limited types* of PHI (demographic information and dates of healthcare) to a business associate or an institutionally related foundation for fundraising purposes, provided the CE has included a fundraising statement in its notice of privacy practices.

```
permitted_by_164_514_f(ARGS, ...) :-
    is_covered_entity(p1),
    (is_business_associate_of(p2, p1) ; is_related_foundation(p2, p1)),
    is_phi(t), msg_contains(m, q, t),
    (is_demographic_info(t) ; is_healthcare_dates(t)),  // limited types
    is_for_fundraising(u),
    has_given_fundraising_notice(p1, p2, q, t, u).
```

Listing 72: Fundraising disclosure (§514(f))

21.6 Health Plan Underwriting (§514(g))

Section 514(g) is a **negative norm**: it restricts health plans that receive PHI for underwriting from using it for other purposes, unless required by law. Since our system defaults to denial (closed-world assumption), this restriction is naturally enforced — a disclosure for a non-underwriting purpose would need to find a *different* positive pathway to be permitted.

22 Formalization of §164.524 — Individual Access

22.1 Overview

Section 164.524 establishes the individual's **right of access** to their own PHI. When a patient requests access to their medical records, the covered entity *must* provide it — this is a **required** disclosure, not merely a permitted one. The key output predicate, `required_by_164_524`, feeds into §502(a)(2)(i), which covers disclosures required to the individual.

22.2 The Right of Access

The core rule is:

```
required_by_164_524(ARGS) :-
    is_covered_entity(p1),
    (p2 = q ; is_personal_representative(p2, q)),  // self or rep
    is_phi(t),
    msg_contains(m, q, t),
    is_access_request(p2, p1, q, t),                // oracle
    is_in_designated_record_set(p1, q, t),           // oracle
```

```
8       !may_deny_without_review_524a2(p2, p1, t),      // no unreviewable denial
9       !may_deny_with_review_524a3(p2, p1, t),          // no reviewable denial
10      known_purpose(u).
```

Listing 73: Right of access (§524) — from hipaa_164_524.dl

The rule says: access is required when the requester is the individual (or their personal representative), the information is in a designated record set, an access request has been made, and no denial ground applies. The two negated predicates check for grounds to deny access.

22.3 Denial Grounds Without Review (§524(a)(2))

A covered entity may deny access *without* providing an opportunity for review in five situations:

(a)(2)(i) The information is: psychotherapy notes, compiled for a legal proceeding, subject to CLIA lab restrictions, or exempt under 42 CFR 493.

(a)(2)(ii) The requester is an inmate and access would jeopardize health, safety, or custody.

(a)(2)(iii) The information was created for research that includes treatment, the individual agreed to denial of access during the research, and was informed access would be reinstated.

(a)(2)(iv) The information is subject to the Privacy Act (5 U.S.C. 552a) and may be denied under that Act.

(a)(2)(v) The information was obtained under a promise of confidentiality and disclosure would reveal the source.

22.4 Denial Grounds With Review (§524(a)(3))

Three additional denial grounds exist, but the individual must be offered a **review** by an independent licensed health care professional:

(a)(3)(i) A licensed professional determines access would endanger the life or safety of the individual or another person.

(a)(3)(ii) The information references another person, and a licensed professional determines access would cause substantial harm to that person.

(a)(3)(iii) A personal representative requests access, and a licensed professional determines providing it would cause substantial harm to the individual.

All denial grounds use oracle predicates (e.g., `determines_access_would_endanger`, `determines_likely_to_cause_harm_to_other`), because they involve professional judgment that the Datalog engine cannot compute.

22.5 How §524 Connects to §502

When `required_by_164_524` holds, it feeds into §502(a)(2)(i):

```
1  // In hipaa_164_502.dl:
2  required_by_164_502_a_2_i(ARGS, ...) :-
3      required_by_164_524(ARGS).  // individual access required
```

Listing 74: How §524 feeds into the main permission framework

This makes individual access a `required_by_` pathway (not merely permitted), which means the CE has an affirmative obligation to provide the disclosure.

23 Testing Methodology

23.1 Test Case Structure

Every test case follows a four-file structure within its own directory:

File	Purpose
`scenario_NNN.md`	Human-readable description of the situation, the question, and the expected verdict.
`fact_NNN.dl`	Datalog facts for the scenario: principals, roles, messages, oracle predicates, and `disclosure_attempted`.
`query_NNN.dl`	Entry point that `#include`s the main formalization files and the fact file.
`run_test_case_NNN.sh`	Shell script that runs Soufflé and checks the result.

Table 7: Test case file structure.

23.2 The Run Script Pattern

Each test case has a shell script that runs Soufflé and interprets the output:

```
#!/bin/bash
cd "$(dirname "$0")"
mkdir -p output
souffle -D output query_NNN.dl 2>&1
if [ -s output/is_disclosure_allowed.csv ]; then
    echo "RESULT: ALLOWED"
    head -5 output/is_disclosure_allowed.csv
elif [ -s output/is_disclosure_denied.csv ]; then
    echo "RESULT: DENIED"
    cat output/is_disclosure_denied.csv
else
    echo "RESULT: NO DISCLOSURE ATTEMPTED"
fi
```

Listing 75: Typical test case run script

The key insight: a test case passes if the Soufflé output matches the expected verdict in the scenario file. If the scenario says "DENY" and `is_disclosure_denied.csv` is non-empty (while `is_disclosure_allowed.csv` is empty), the test passes.

23.3 Test Coverage

The formalization includes three sets of test cases:

Systematic test cases (124 cases): Generated to cover every clause of every formalized section. Each test targets a specific rule or exception, ensuring that the rule fires (or correctly does not fire) under the right conditions.

Realistic test cases (25 cases): More complex scenarios that combine multiple HIPAA sections and test edge cases, such as disclosures that traverse multiple permission pathways or trigger multiple exception checks.

GoldCoin test cases (28 cases): Based on real court cases involving HIPAA disputes. These test whether the formalization reaches the same conclusion as the courts. For example, *Michelson v. City of Plainfield* (a citizen requesting employee health insurance details) should be denied because no HIPAA permission pathway applies.

Design Decision

The GoldCoin test cases are particularly valuable because they validate the formalization against real-world legal outcomes, not just our interpretation of the regulation text. When a GoldCoin test case fails, it often reveals a subtle gap in the formalization that synthetic test cases miss.

24 Oracle Predicates — A Complete Reference

24.1 What are Oracle Predicates?

Throughout this document, we have used the term "oracle predicate" to describe predicates that are declared (`.decl`) but never derived by rules. They are populated only by asserting ground facts. The term "oracle" comes from complexity theory — just as a Turing machine with an oracle can query an external source for answers, our Datalog program queries these predicates for contextual information that cannot be derived from the formalization itself.

Oracle predicates are necessary because HIPAA frequently conditions disclosures on:

- **Subjective beliefs**: "the covered entity believes in good faith..."
- **Professional judgment**: "in the professional judgment of the provider..."
- **External legal facts**: "required by law," "authorized by statute..."
- **Contractual states**: "a data use agreement is in place..."
- **Document contents**: "the authorization contains a description of..."

None of these can be computed from the structure of the HIPAA rules alone. They must be provided as input to the compliance checker.

24.2 How Oracle Predicates are Used

An oracle predicate appears in a rule body just like any derived predicate. The difference is purely in how it gets its data:

```
// is_required_by_law is an oracle -- asserted as a fact in the test scenario
.decl is_required_by_law(p1: Principal, p2: Principal,
    q: Principal, t: Attribute, u: Purpose)

// The rule uses it like any other predicate
permitted_by_164_512_a(ARGS,
    $Leaf("164.512(a): required by law")) :-
    is_covered_entity(p1),
```

```
    is_phi(t),
    msg_contains(m, q, t),
    is_required_by_law(p1, p2, q, t, u).  // <- oracle in body
```

Listing 76: Oracle predicate used in a rule

In a test case fact file, you would assert:

```
// In fact_NNN.dl:
is_required_by_law("hospital_A", "state_health_dept",
    "patient_X", "communicable-disease-report", "disease-reporting").
```

Listing 77: Asserting an oracle predicate as a fact

If this fact is present, the §512(a) rule fires. If it is absent, the rule does not fire (closed-world assumption).

24.3 Oracle Predicates by Section

The following table groups all oracle predicates by the HIPAA section that introduces them. This is a reference for test case authors — when writing a test scenario, consult this list to determine which oracle predicates need to be asserted.

§164.502/506 (General Rules and TPO):

- `believes_minimum_necessary` — CE believes disclosure is minimum necessary
- `satisfactory_assurances` — BA has provided adequate safeguards
- `has_authority_to_act` — personal representative authority
- `abuse_exception` — personal representative abuse exception
- `believes_unlawful_conduct` — whistleblower belief
- `secretary_investigation_authorized` — HHS Secretary investigation
- `has_obtained_consent_506b` — TPO consent obtained
- `known_purpose` — grounding helper for unrestricted purpose variables

§164.508 (Authorization):

- `obtained_authorization_164_508` — valid authorization obtained
- `is_valid_authorization` — authorization document is valid
- `is_defective_authorization` — authorization has defects
- `face_to_face` — marketing via face-to-face communication
- `promotional_gift_of_nominal_value` — marketing with nominal gift
- `legal_defense_purpose` — CE defending itself in legal proceeding

§164.510 (Agree/Object):

- `has_not_objected_to_directory` — individual did not object to directory
- `is_directory_request_by_name` — someone asked for individual by name
- `consistent_with_prior_preference` — consistent with prior expressed preference
- `believes_in_best_interest_directory` — CE believes directory disclosure is in best interest
- `is_family_member`, `is_close_personal_friend`, `is_identified_by_individual` — relationship oracles

- `relevant_to_involvement` — info relevant to care involvement
- `has_obtained_agreement_510b2` — individual has agreed
- `professional_judgment_best_interest_510b3` — professional judgment for incapacitated individual

§164.512 (Without Authorization):

- `is_required_by_law` — disclosure required by another law
- `is_authorized_by_law_for_purpose` — recipient authorized by law
- `believes_victim_of_abuse` — CE believes individual is abuse victim
- `has_court_order` — court order received
- `has_lawful_process_with_assurance` — subpoena with satisfactory assurance
- `has_irb_or_privacy_board_waiver` — research waiver approved
- `believes_necessary_to_lessen_threat` — serious threat determination
- `deemed_necessary_for_mission` — military necessity determination
- `authorized_for_workers_comp` — workers' compensation authorization

§164.514 (Other Requirements):

- `expert_determination_deidentified` — expert de-identification determination
- `safe_harbor_deidentified` — safe harbor de-identification
- `identifies_workforce_needing_phi`, `reasonably_limits_phi_access`, etc. — minimum necessary implementation oracles
- `has_limited_data_use_agreement` — data use agreement in place
- `has_given_fundraising_notice` — fundraising notice provided

§164.524 (Individual Access):

- `is_access_request` — individual has requested access
- `is_in_designated_record_set` — PHI is in a designated record set
- `compiled_for_legal_proceeding` — information compiled for litigation
- `determines_access_would_endanger` — professional determination of endangerment
- `agreed_to_denial_of_access` — research participant agreed to access denial

25 Conclusion

This document has provided a comprehensive walkthrough of the HIPAA Privacy Rule formalization in Soufflé Datalog, covering seven sections: §164.502, §164.506, §164.508, §164.510, §164.512, §164.514, and §164.524. The key takeaways are:

1. **Architecture**: The checker is modularly organized into types, hierarchies, macros, section-specific rules, a top-level aggregator, and test fact databases.

2. **Explanation trees**: The `Expl` ADT provides structured, auditable provenance for every compliance decision.

3. **Hierarchies**: Role, attribute, and purpose hierarchies with transitive closure enable subsumption (a doctor is a provider is a covered entity).

4. **Stratified negation**: Used for negative norms (§508 blocking §506), exceptions, exclusions, denial grounds (§524), and denial tracking.

5. **Oracle predicates**: Contextual conditions — professional judgment, legal determinations, contractual states — are modeled as oracle predicates, providing a clean interface between the formal rules and real-world context.
6. **Grounding**: Helper relations (`known_purpose`, `disclosure_attempted`) solve the grounding requirement for unrestricted variables.
7. **Testing**: 158 test cases (105 systematic, 25 realistic, 28 GoldCoin court cases) validate the formalization against expected outcomes and real legal decisions.
8. **Verification**: The v2 formalization was verified clause-by-clause against the actual eCFR regulatory text through three iterative rounds. The maximal revelation principle was checked across all 120+ rules, ensuring that conjunctions, disjunctions, and negations are properly decomposed. 26 fixes were applied including: §502(a)(5) prohibitions on genetic underwriting and sale of PHI, §508(a)(4) sale authorization, authorization validity checks, correctional release guards, DNA/dental prohibition on law enforcement identification, and 5 disjunction splits for better provenance.

To extend the formalization to additional HIPAA sections (e.g., §164.520, §164.522, §164.528, §164.530), follow the same pattern:

1. Read the HIPAA regulation text and the PDF formal notation.
2. Remove any stub declarations for the section from `hipaa_stubs.dl`.
3. Write Soufflé rules following the translation patterns described here.
4. Create test cases with scenario, fact, query, and run script files.
5. Run the checker and verify expected outcomes.

Appendix A Complete File: hipaa_types.dl

```
// Preprocessor macros for the 6-tuple action signature
#define ARGS p1, p2, q, m, t, u
#define DECL_ARGS p1:Principal, p2:Principal, q:Principal, m:Message, t:Attribute, u:Purpose

// Primitive domain subtypes
.type Principal <: symbol
.type Role <: symbol
.type Attribute <: symbol
.type Purpose <: symbol
.type Message <: symbol
.type TimePoint <: number
.type Rel <: symbol

// ADT: Explanation tree
.type Expl = Leaf  { reason: symbol }
           | Step1 { reason: symbol, sub1: Expl }
           | Step2 { reason: symbol, sub1: Expl, sub2: Expl }
           | Step3 { reason: symbol, sub1: Expl, sub2: Expl, sub3: Expl }
```

Appendix B Complete File: hipaa_top.dl

```
// Top-level: Is this disclosure allowed?
.decl is_disclosure_allowed(DECL_ARGS, e: Expl)
is_disclosure_allowed(ARGS, E) :-
    permitted_by_164_502(ARGS, E).

// Denial reporting
.decl disclosure_attempted(DECL_ARGS)
.decl is_disclosure_denied(DECL_ARGS)
is_disclosure_denied(ARGS) :-
    disclosure_attempted(ARGS),
    !is_disclosure_allowed(ARGS, _).

// Output directives
.output is_disclosure_allowed
.output is_disclosure_denied
.output disclosure_attempted
.output permitted_by_164_502
.output permitted_by_164_506
.output minimum_necessary_satisfied
.output is_personal_representative
.output excluded_164_502_b_2
```

Appendix C System Architecture Overview

This section describes the full ComplianceGPT system architecture introduced in v3. The system connects natural-language compliance questions to formal Soufflé Datalog engines across six regulations, through a four-strategy pipeline.

C.1 High-Level Pipeline

Every compliance query follows the same four-stage pipeline regardless of which strategy is used:

1. **Input:** A natural-language question (e.g., "Can a hospital share patient records with a referring physician for treatment?").
2. **Scenario extraction (LLM1):** The question is parsed into a structured `DatalogScenario` JSON object: sender, sender_role, receiver, receiver_role, subject, subject_category, attribute, purpose, and oracle predicates.
3. **Compliance check:** Depending on the active strategy:
   - **Baseline:** A single LLM call with a regulation-aware system prompt.
   - **RAG:** Hybrid BM25+semantic retrieval from a regulation-specific knowledge base, then LLM reasoning over retrieved text.
   - **Formal:** Soufflé Datalog engine for the active regulation.
   - **Agentic:** Multi-agent pipeline (Agents A–E) with self-correction.
4. **Explanation (LLM2):** The verdict and proof tree are translated back to plain English.

C.2 Components and Roles

Component	File(s)	Role
`ModelClient`	`connector/model_client.py`	Unified interface to Anthropic, OpenAI, Google, Ollama, HuggingFace. Routes `complete()` calls to the right provider. Retries on empty responses (3 attempts for Ollama).
`Settings`	`connector/settings.py`	Persistent key-value store (`.compliancegpt_settings.json`). Holds `active_regulation`, LLM provider/model, Soufflé path, RAG parameters.
`LLM1Extractor`	`connector/llm1_extractor.py`	Parses natural-language question into `DatalogScenario`. Falls back to safe defaults for any missing required field (critical for small local models).
`LLM2Explainer`	`connector/llm2_explainer.py`	Translates Soufflé output (verdict + ADT explanation tree) into a plain-English paragraph with section citations.
`BaselineStrategy`	`app/strategies/baseline.py`	Single LLM call with regulation-aware system prompt. Returns `PERMITTED` or `DENIED`.
`RAGStrategy`	`app/strategies/rag.py`	Loads regulation-specific knowledge base, builds BM25 + sentence-transformer indices, retrieves top-k chunks, calls LLM with context.
`FormalStrategy`	`app/strategies/formal.py`	Calls LLM1, generates Datalog facts, invokes Soufflé via subprocess, parses output CSV, calls LLM2 for explanation. Supports all 6 regulation engines via `REGULATION_FILES` dict.
`AgenticStrategy`	`app/strategies/agentic.py`	Multi-agent pipeline: Entity extraction (A), Section mapping (B), Validator (C), Soufflé, Self-correction (D), Explainer (E).
`batch_runner.py`	`app/batch_runner.py`	CLI batch evaluator. Accepts `-regulation` to set active regulation before running. Handles hipaa_qa, goldcoin, privacyci, and generic CSV formats.
`run_benchmark.py`	`run_benchmark.py`	Meta-runner iterating over all models × strategies × datasets. Passes `-regulation` to each `batch_runner.py` call. Prints accuracy table.
`streamlit_app.py`	`app/streamlit_app.py`	Interactive web UI. Regulation selector (6 regulations), strategy selector (4 strategies), live query interface, batch evaluation page.

C.3 Multi-Regulation Strategy

The key architectural insight is the **bridge interface**: all six Soufflé engines accept the same input predicate (`disclosure_attempted(p1,p2,q,m,t,u)`) and produce the same output predicates (`is_disclosure_allowed`, `is_disclosure_denied`). This means:

- `FormalStrategy` can select the correct `.dl` file from `REGULATION_FILES` based on `active_regulation` without any other changes.
- `LLM1Extractor` uses the same `DatalogScenario` schema for all regulations.
- Test CSVs for all 6 regulations use the same column format (`question`, `answer`).

The `_adapt_facts()` method in `FormalStrategy` strips HIPAA-specific predicates (e.g., `has_baa`, `is_organized_health_care_arrangement`) when running non-HIPAA engines, so the same LLM1 output can be used across regulations.

C.4 Active Regulation Routing

The `active_regulation` setting controls which engine and knowledge base are used:

```
REGULATION_FILES = {
    "hipaa": [
        "hipaa_types.dl", "hipaa_hierarchies.dl", "hipaa_macros.dl",
        "hipaa_stubs.dl", "hipaa_164_506.dl", "hipaa_164_502.dl",
        "hipaa_164_508.dl", "hipaa_164_510.dl", "hipaa_164_512.dl",
        "hipaa_164_514.dl", "hipaa_164_524.dl", "hipaa_top.dl",
    ],
    "gdpr":  ["gdpr_top.dl"],
    "glba":  ["glba_top.dl"],
    "ccpa":  ["ccpa_top.dl"],
    "coppa": ["coppa_top.dl"],
    "sox":   ["sox_top.dl"],
}
```

Listing 78: REGULATION_FILES dict in app/strategies/formal.py

Non-HIPAA regulations use only their `*_top.dl` file because that file uses `#include` directives to pull in its own types, hierarchies, and rules.

Appendix D Multi-Regulation Formalization (v3)

D.1 Design Principles for Non-HIPAA Regulations

All non-HIPAA regulations follow the same structural pattern as HIPAA:

1. **Types file** (`*_types.dl`): Declares `.type` aliases for `Principal`, `Role`, `Attribute`, `Purpose` specific to the regulation. Reuses the same ADT `Expl` type for explanation trees.
2. **Hierarchies file** (`*_hierarchies.dl`): Declares the role, attribute, and purpose hierarchies for the regulation. All use the same predicate names (`role_isa`, `attr_isa`, `purpose_isa`) for uniformity.
3. **Rules file(s)**: One file per major section (e.g., `gdpr_art6.dl`, `sox_302.dl`).

4. **Top file** (`*_top.dl`): Includes all the above, declares the bridge predicates, and has `.output` directives.

Warning

No `.input` directives in top files. Soufflé `.input` directives tell the engine to load from `.facts` files on disk. All ComplianceGPT engines provide facts via inline Datalog rules (included via `#include`), not as external files. Any `.input` directive will cause `Cannot open fact file` errors. HIPAA works correctly because its `hipaa_top.dl` has no `.input` directives. All non-HIPAA top files must also have no `.input` directives.

D.2 GDPR — General Data Protection Regulation

Files: `gdpr_types.dl`, `gdpr_hierarchies.dl`, `gdpr_art6.dl`, `gdpr_art17.dl`, `gdpr_top.dl`

Formalized sections:

- **Article 6 — Lawful basis for processing:** Six lawful bases (consent, contract, legal obligation, vital interests, public task, legitimate interests). The engine checks which basis applies to the disclosure scenario.
- **Article 17 — Right to erasure ("right to be forgotten"):** Conditions under which a data subject can require erasure of personal data, and exceptions that override the right.

Key attributes: `personal-data`, `sensitive-data`, `health-data`, `biometric-data`
Key purposes: `consent-given`, `contract-performance`, `legal-obligation`, `legitimate-interests`
Key roles: `controller`, `processor`, `data-subject`, `supervisory-authority`

```
permitted_by_gdpr_art6_1_a(ARGS,
    $Leaf("Art.6(1)(a): processing based on consent of data subject")) :-
    has_role(p1, "controller"),
    (attr_isa(t, "personal-data") ; t = "personal-data"),
    msg_contains(m, q, t),
    (u = "consent-given" ; purpose_isa(u, "consent-given")),
    has_given_consent(q, p1, t).
```

Listing 79: GDPR Article 6(1)(a) lawful basis (consent) rule

D.3 GLBA — Gramm-Leach-Bliley Act

Files: `glba_types.dl`, `glba_hierarchies.dl`, `glba_313_14.dl`, `glba_top.dl`

Formalized sections:

- **§313.14 — Exceptions to notice and opt-out requirements:** Conditions under which a financial institution may share NPI (non-public personal information) with non-affiliated third parties without providing opt-out notice.

Key attributes: `npi` (non-public personal information), `financial-record`, `credit-info`
Key purposes: `service-provider`, `joint-marketing`, `required-by-law-glba`, `fraud-prevention`
Key roles: `financial-institution`, `affiliated-party`, `non-affiliated-third-party`, `consumer`

```
permitted_by_glba_313_14_a(ARGS,
    $Leaf("313.14(a): service provider exception")) :-
    has_role(p1, "financial-institution"),
    (attr_isa(t, "npi") ; t = "npi"),
    msg_contains(m, q, t),
    has_written_agreement(p1, p2),
    (u = "service-provider" ; purpose_isa(u, "service-provider")).
```

Listing 80: GLBA §313.14(a) exception for service providers

D.4 CCPA — California Consumer Privacy Act

Files: `ccpa_types.dl`, `ccpa_hierarchies.dl`, `ccpa_1798_100.dl`, `ccpa_1798_120.dl`, `ccpa_top.dl`

Formalized sections:

- **§1798.100 — Right to know:** California consumers' right to access their personal information held by a business.
- **§1798.120 — Right to opt out of sale:** Consumers' right to direct a business not to sell personal information, and the business's obligations.

Key attributes: `personal-information`, `sensitive-personal-information`, `biometric-info`
Key purposes: `business-purpose`, `sale-of-data`, `service-provider-purpose`
Key roles: `business`, `service-provider`, `consumer`, `third-party`

Required field defaults: Ollama small models sometimes omit `subject_category` from JSON output. The `LLM1Extractor` fills in `"adult"` as the default, which correctly enables CCPA rules (CCPA applies to adults; children under 16 require opt-in consent).

D.5 COPPA — Children's Online Privacy Protection Act

Files: `coppa_types.dl`, `coppa_hierarchies.dl`, `coppa_312_4.dl`, `coppa_312_5.dl`, `coppa_top.dl`

Formalized sections:

- **§312.4 — Notice:** Requirements for operators to provide direct notice to parents before collecting personal information from children under 13.
- **§312.5 — Verifiable parental consent:** Methods and standards for obtaining verifiable parental consent; educational context exception.

Key attributes: `child-data`, `contact-info`, `geolocation-data`, `photo-video`
Key purposes: `parental-consent-given`, `educational-purpose`, `internal-use-only`
Key roles: `operator`, `parent`, `child-under-13`, `educational-institution`

```
permitted_by_coppa_312_5_educational(ARGS,
    $Leaf("312.5: educational context, school as agent of parent")) :-
    has_role(p1, "operator"),
    (attr_isa(t, "child-data") ; t = "child-data"),
    is_child_under_13(q),
    is_educational_context(p1, q),
    is_internal_use_only(m).
```

Listing 81: COPPA §312.5 educational context exception

D.6 SOX — Sarbanes-Oxley Act

Files: `sox_types.dl`, `sox_hierarchies.dl`, `sox_302.dl`, `sox_404.dl`, `sox_top.dl`

Formalized sections:

- **§302 — Corporate responsibility for financial reports:** CEO and CFO must personally certify the accuracy of periodic reports. The engine checks whether both required certifications are present and the signing officers have authority.
- **§404 — Management assessment of internal controls:** Management must assess internal controls over financial reporting; auditor must attest to that assessment. The engine checks whether the assessment exists, is complete, and has been attested.

Key attributes: `financial-report`, `material-weakness`, `internal-control-assessment`
Key purposes: `periodic-reporting`, `internal-control-reporting`, `auditor-attestation`
Key roles: `public-issuer`, `ceo`, `cfo`, `registered-auditor`, `audit-committee`

```
permitted_by_sox_302(ARGS,
    $Step2("302: CEO and CFO certification of periodic report",
           $Leaf("CEO certification present"),
           $Leaf("CFO certification present"))) :-
    has_role(p1, "public-issuer"),
    is_public_issuer(p1),
    (attr_isa(t, "financial-report") ; t = "financial-report"),
    has_ceo_cfo_certification(p1, m),
    !has_material_weakness(p1).
```

Listing 82: SOX §302 dual-certification rule

D.7 Unified Bridge Interface

All six regulation engines share this interface. The `FormalStrategy` generates facts in this format regardless of which regulation is active:

```
// Input fact (generated by LLM1Extractor for any regulation)
.decl disclosure_attempted(
    p1: Principal, p2: Principal, q: Principal,
    m: Message, t: Attribute, u: Purpose)

// Output relations (same predicate names for all regulations)
.decl is_disclosure_allowed(
    p1: Principal, p2: Principal, q: Principal,
    m: Message, t: Attribute, u: Purpose, e: Expl)

.decl is_disclosure_denied(
    p1: Principal, p2: Principal, q: Principal,
    m: Message, t: Attribute, u: Purpose)

// Denial rule (identical in all top files)
is_disclosure_denied(ARGS) :-
    disclosure_attempted(ARGS),
    !is_disclosure_allowed(ARGS, _).
```

```
19
20  .output is_disclosure_allowed
21  .output is_disclosure_denied
```

Listing 83: Bridge interface — identical across all 6 regulation engines

Appendix E RAG Knowledge Base Design

The RAG strategy uses a hybrid BM25 + semantic retrieval over regulation-specific knowledge bases. The knowledge base is selected based on `active_regulation`.

E.1 Knowledge Base Files

Regulation	File	Format and Contents
HIPAA	`data/ecfr_parsed/*.txt`	Per-section plain text files parsed from eCFR Title 45 XML. One file per CFR section (§164.501–164.534).
GDPR	`data/gdpr_rag_db.csv`	CSV with columns: `id`, `section_number`, `original_text`. Article-level text.
GLBA	`data/glba_rag_db.xml`	XML regulation text. Loaded element-by-element; elements with >50 chars of text are indexed.
CCPA	`data/ccpa_rag_db.csv`	CSV with columns: `id`, `section_number`, `original_text`. California Civil Code §1798.x.
COPPA	`data/coppa_rag_db.xml`	XML regulation text for 16 CFR Part 312.
SOX	`data/sox_rag_db.csv`	CSV with columns: `id`, `section_number`, `original_text`. SOX §302/404 provisions.

Table 9: RAG knowledge base files per regulation.

E.2 Hybrid Retrieval Algorithm

The hybrid retrieval score is:

$$\text{score}(d, q) = \alpha \cdot s_{\text{sem}}(d, q) + (1 - \alpha) \cdot s_{\text{BM25}}(d, q)$$

where both scores are min-max normalized to $[0, 1]$ before combination. The default $\alpha = 0.5$ gives equal weight to semantic similarity (sentence-transformer `all-MiniLM-L6-v2`) and lexical overlap (BM25Okapi). This is configurable in Settings.

Why hybrid matters for legal text:

- BM25 is strong on exact statutory terms ("verifiable parental consent", "NPI", "material weakness").
- Semantic search handles paraphrase ("sharing customer data with partners" → "disclosure to non-affiliated third parties").
- Neither alone achieves the same recall as the combination.

Appendix F Benchmarking and Evaluation

F.1 QA Datasets

Each regulation has a dedicated QA dataset in `data/`:

File	Rows	Description
`hipaa_qa.csv`	90	HIPAA privacy questions (question, answer: PERMITTED/-DENIED)
`goldcoin.csv`	varies	GoldCoin HIPAA benchmark
`hipaa_privacyci.csv`	varies	PrivacyCI narrative benchmark
`gdpr_qa.csv`	10	GDPR compliance questions
`glba_qa.csv`	10	GLBA financial privacy questions
`ccpa_qa.csv`	10	CCPA consumer privacy questions
`coppa_qa.csv`	10	COPPA children's privacy questions
`sox_qa.csv`	10	SOX corporate governance questions

Table 10: QA datasets for all six regulations.

F.2 Running the Benchmark

The benchmark runner (`run_benchmark.py`) automates evaluation across all combinations:

```
# All 6 regulations x all 4 strategies x all installed Ollama models
python run_benchmark.py --regulation all --limit 5

# HIPAA only, baseline + formal, specific model
python run_benchmark.py --regulation hipaa --strategies baseline formal \
    --models llama3:latest --limit 10

# Full run, resume on restart
python run_benchmark.py --regulation all --skip-existing
```

F.3 Accuracy Metrics

The benchmark computes per-run accuracy as:

$$\text{Accuracy} = \frac{|\{q : \text{predicted\_verdict}(q) = \text{ground\_truth}(q)\}|}{|\{q : q \text{ is labeled}\}|}$$

Unlabeled rows (no ground truth) are excluded from accuracy calculation but are still evaluated. Results are saved as CSVs in `results/<timestamp>/` with columns: `row_id`, `question`, `ground_truth`, `strategy`, `model`, `verdict`, `match` (Y/N), `citations`, `latency_s`, `llm_answer`.

F.4 Confirmed Working Matrix

As of April 2026, all 24 combinations (6 regulations $\times$ 4 strategies) run without errors using `llama3:latest`:

Regulation	Baseline	RAG	Formal	Agentic
HIPAA	✓	✓	✓	✓
GDPR	✓	✓	✓	✓
GLBA	✓	✓	✓	✓
CCPA	✓	✓	✓	✓
COPPA	✓	✓	✓	✓
SOX	✓	✓	✓	✓

Table 11: All 24 regulation × strategy combinations confirmed working.

Appendix G Known Bugs Fixed (v3)

This section documents the bugs discovered during multi-regulation integration and their fixes.

G.1 Bug 1: `.input` Directives in Non-HIPAA Top Files

Symptom: Soufflé exits with `Error loading disclosure_attempted data: Cannot open fact file disclosure_attempted.facts` for all non-HIPAA regulations.

Root cause: The `.input relation_name` directive tells Soufflé to load tuples from a `relation_name.facts` file on disk. HIPAA works because `hipaa_top.dl` has no `.input` directives — facts come from inline Datalog rules via `#include "hipaa_facts.dl"`. The initial non-HIPAA top files had `.input` directives copied from an older template.

Fix: Removed all `.input` directives from `gdpr_top.dl`, `glba_top.dl`, `ccpa_top.dl`, `coppa_top.dl`, and `sox_top.dl`. The `.decl` declarations (type signatures) are kept; only the `.input` directives are removed.

G.2 Bug 2: Missing `subject_category` Field (CCPA Crash)

Symptom: CCPA runs produced empty output with `ERROR` verdict. Full error: `TypeError: DatalogScenario.__init__() missing 1 required positional argument: 'subject_category'`.

Root cause: Small local models (llama3) sometimes omit fields from their JSON output. `DatalogScenario` requires `subject_category` (used to determine if CCPA's child-under-16 opt-in rule applies).

Fix: Added `_REQUIRED_DEFAULTS` dictionary in `connector/llm1_extractor.py extract()` method. Before constructing `DatalogScenario`, any missing required field is filled with a safe default:

```
_REQUIRED_DEFAULTS = {
    "subject_category": "adult",
    "sender":           "entity_a",
    "sender_role":      "individual",
    "receiver":         "entity_b",
    "receiver_role":    "individual",
    "subject":          "subject_a",
    "attribute":        "medical-record",
    "purpose":          "healthcare-operations",
    "message_id":       "msg_001",
```

```
}
for k, v in _REQUIRED_DEFAULTS.items():
    if k not in data or not data[k]:
        data[k] = v
```

Listing 84: Required field defaults in llm1_extractor.py

G.3 Bug 3: `rank-bm25` Not Installed

Symptom: `ModuleNotFoundError: No module named 'rank_bm25'` when running any RAG strategy.

Fix: `pip install rank-bm25`. Added to `requirements.txt`.

G.4 Bug 4: Ollama Empty Response in Agentic Strategy

Symptom: Agentic strategy returned `ERROR` for some regulations when using Ollama models. The `_ollama()` method in `model_client.py` did not retry on empty content.

Fix 1: Added a 3-attempt retry loop in `connector/model_client.py _ollama()` for empty content responses.

Fix 2: Added `response_format="json"` to the Ollama API call in `app/strategies/agentic.py` to reduce the likelihood of malformed responses.